Package 'DESeq2'

March 24, 2025

Type Package

Title Differential gene expression analysis based on the negative binomial distribution

Version 1.46.0

Maintainer Michael Love <michaelisaiahlove@gmail.com>

Description Estimate variance-mean dependence in count data from high-throughput sequencing assays and test for differential expression based on a model using the negative binomial distribution.

License LGPL (>= 3)

VignetteBuilder knitr, rmarkdown

Imports BiocGenerics (>= 0.7.5), Biobase, BiocParallel, matrixStats, methods, stats4, locfit, ggplot2 (>= 3.4.0), Rcpp (>= 0.11.0), MatrixGenerics

Depends S4Vectors (>= 0.23.18), IRanges, GenomicRanges, SummarizedExperiment (>= 1.1.6)

Suggests testthat, knitr, rmarkdown, vsn, pheatmap, RColorBrewer, apeglm, ashr, tximport, tximeta, tximportData, readr, pbapply, airway, pasilla (>= 0.2.10), glmGamPoi, BiocManager

LinkingTo Rcpp, RcppArmadillo

URL https://github.com/thelovelab/DESeq2

biocViews Sequencing, RNASeq, ChIPSeq, GeneExpression, Transcription, Normalization, DifferentialExpression, Bayesian, Regression, PrincipalComponent, Clustering, ImmunoOncology

RoxygenNote 7.3.1

Encoding UTF-8

git_url https://git.bioconductor.org/packages/DESeq2

git_branch RELEASE_3_20

git_last_commit 4887eb4

git_last_commit_date 2024-10-29

Repository Bioconductor 3.20

Date/Publication 2025-03-24

Author Michael Love [aut, cre],
Constantin Ahlmann-Eltze [ctb],
Kwame Forbes [ctb],
Simon Anders [aut, ctb],
Wolfgang Huber [aut, ctb],
RADIANT EU FP7 [fnd],
NIH NHGRI [fnd],
CZI [fnd]

Contents

DESeq2-package . . . 3
coef.DESeqDataSet . . . 4
collapseReplicates . . . 4
counts,DESeqDataSet-method . . . 6
DESeq . . . 7
DESeqDataSet-class . . . 10
DESeqResults-class . . . 12
DESeqTransform-class . . . 12
design,DESeqDataSet-method . . . 13
dispersionFunction . . . 14
dispersions . . . 15
estimateBetaPriorVar . . . 15
estimateDispersions,DESeqDataSet-method . . . 16
estimateDispersionsGeneEst . . . 19
estimateSizeFactors,DESeqDataSet-method . . . 21
estimateSizeFactorsForMatrix . . . 23
fpkm . . . 24
fpm . . . 25
integrateWithSingleCell . . . 27
lfcShrink . . . 28
makeExampleDESeqDataSet . . . 30
nbinomLRT . . . 31
nbinomWaldTest . . . 33
normalizationFactors . . . 35
normalizeGeneLength . . . 36
normTransform . . . 37
plotCounts . . . 37
plotDispEsts . . . 38
plotMA . . . 39
plotPCA . . . 41
plotSparsity . . . 42
priorInfo . . . 43
replaceOutliers . . . 43
results . . . 44
rlog . . . 50
show,DESeqResults-method . . . 52
sizeFactors,DESeqDataSet-method . . . 53
summary,DESeqResults-method . . . 53
unmix . . . 54
varianceStabilizingTransformation . . . 55
vst . . . 57

Index **59**

DESeq2-package *DESeq2 package for differential analysis of count data*

Description

The DESeq2 package is designed for normalization, visualization, and differential analysis of high-dimensional count data. It makes use of empirical Bayes techniques to estimate priors for log fold change and dispersion, and to calculate posterior estimates for these quantities.

Details

The main functions are:

- `DESeqDataSet` - build the dataset, see tximeta & tximport packages for preparing input
- `DESeq` - perform differential analysis
- `results` - build a results table
- `lfcShrink` - estimate shrunken LFC (posterior estimates) using apeglm & ashr pakges
- `vst` - apply variance stabilizing transformation, e.g. for PCA or sample clustering
- Plots, e.g.: `plotPCA`, `plotMA`, `plotCounts`

For detailed information on usage, see the package vignette, by typing `vignette("DESeq2")`, or the workflow linked to on the first page of the vignette.

All software-related questions should be posted to the Bioconductor Support Site:

https://support.bioconductor.org

The code can be viewed at the GitHub repository, which also lists the contributor code of conduct:

https://github.com/mikelove/tximport

Author(s)

Michael Love, Wolfgang Huber, Simon Anders

References

Love, M.I., Huber, W., Anders, S. (2014) Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. Genome Biology, 15:550. https://doi.org/10.1186/s13059-014-0550-8

See Also

Useful links:

- https://github.com/thelovelab/DESeq2

coef.DESeqDataSet *Extract a matrix of model coefficients/standard errors*

Description

Note: results tables with log2 fold change, p-values, adjusted p-values, etc. for each gene are best generated using the `results` function. The `coef` function is designed for advanced users who wish to inspect all model coefficients at once.

Usage

```
## S3 method for class 'DESeqDataSet'
coef(object, SE = FALSE, ...)
```

Arguments

`object`	a DESeqDataSet returned by `DESeq`, `nbinomWaldTest`, or `nbinomLRT`.
`SE`	whether to give the standard errors instead of coefficients. defaults to FALSE so that the coefficients are given.
`...`	additional arguments

Details

Estimated model coefficients or estimated standard errors are provided in a matrix form, number of genes by number of parameters, on the log2 scale. The columns correspond to columns of the model matrix for final GLM fitting, i.e., `attr(dds, "modelMatrix")`.

Author(s)

Michael Love

Examples

```
dds <- makeExampleDESeqDataSet(m=4)
dds <- DESeq(dds)
coef(dds)[1,]
coef(dds, SE=TRUE)[1,]
```

collapseReplicates *Collapse technical replicates in a RangedSummarizedExperiment or DESeqDataSet*

Description

Collapses the columns in `object` by summing within levels of a grouping factor `groupby`. The purpose of this function is to sum up read counts from technical replicates to create an object with a single column of read counts for each sample. This function will issue a warning if there are other assays other than `"counts"`, see details below in 'Value'.

Usage

```
collapseReplicates(object, groupby, run, renameCols = TRUE)
```

Arguments

object	A RangedSummarizedExperiment or DESeqDataSet
groupby	a grouping factor, as long as the columns of object
run	optional, the names of each unique column in object. if provided, a new column runsCollapsed will be added to the colData which pastes together the names of run
renameCols	whether to rename the columns of the returned object using the levels of the grouping factor

Details

Note: by "technical replicates", we mean multiple sequencing runs of the same library, in constrast to "biological replicates" in which multiple libraries are prepared from separate biological units. Optionally renames the columns of returned object with the levels of the grouping factor. Note: this function is written very simply and can be easily altered to produce other behavior by examining the source code.

Value

the object with as many columns as levels in groupby. This object has "counts" data which is summed from the various columns which are grouped together, and the colData is subset using the first column for each group in groupby. Other assays are dropped, as it is not unambiguous the correct form of combination, and a warning is printed if they are present, so the user is aware they should take care of such assays manually.

Examples

```
dds <- makeExampleDESeqDataSet(m=12)

# make data with two technical replicates for three samples
dds$sample <- factor(sample(paste0("sample",rep(1:9, c(2,1,1,2,1,1,2,1,1)))))
dds$run <- paste0("run",1:12)

ddsColl <- collapseReplicates(dds, dds$sample, dds$run)

# examine the colData and column names of the collapsed data
colData(ddsColl)
colnames(ddsColl)

# check that the sum of the counts for "sample1" is the same
# as the counts in the "sample1" column in ddsColl
matchFirstLevel <- dds$sample == levels(dds$sample)[1]
stopifnot(all(rowSums(counts(dds[,matchFirstLevel])) == counts(ddsColl[,1])))
```

counts,DESeqDataSet-method

Accessors for the 'counts' slot of a DESeqDataSet object.

Description

The counts slot holds the count data as a matrix of non-negative integer count values, one row for each observational unit (gene or the like), and one column for each sample.

Usage

```
## S4 method for signature 'DESeqDataSet'
counts(object, normalized = FALSE, replaced = FALSE)

## S4 replacement method for signature 'DESeqDataSet,matrix'
counts(object) <- value
```

Arguments

object	a DESeqDataSet object.
normalized	logical indicating whether or not to divide the counts by the size factors or normalization factors before returning (normalization factors always preempt size factors)
replaced	after a DESeq call, this argument will return the counts with outliers replaced instead of the original counts, and optionally normalized. The replaced counts are stored by DESeq in assays(object)[['replaceCounts']].
value	an integer matrix

Author(s)

Simon Anders

See Also

sizeFactors, normalizationFactors

Examples

```
dds <- makeExampleDESeqDataSet(m=4)
head(counts(dds))

dds <- estimateSizeFactors(dds) # run this or DESeq() first
head(counts(dds, normalized=TRUE))
```

DESeq *Differential expression analysis based on the Negative Binomial (a.k.a. Gamma-Poisson) distribution*

Description

This function performs a default analysis through the steps:

1. estimation of size factors: estimateSizeFactors
2. estimation of dispersion: estimateDispersions
3. Negative Binomial GLM fitting and Wald statistics: nbinomWaldTest

For complete details on each step, see the manual pages of the respective functions. After the DESeq function returns a DESeqDataSet object, results tables (log2 fold changes and p-values) can be generated using the results function. Shrunken LFC can then be generated using the lfcShrink function. All support questions should be posted to the Bioconductor support site: http://support.bioconductor.org.

Usage

```
DESeq(
  object,
  test = c("Wald", "LRT"),
  fitType = c("parametric", "local", "mean", "glmGamPoi"),
  sfType = c("ratio", "poscounts", "iterate"),
  betaPrior,
  full = design(object),
  reduced,
  quiet = FALSE,
  minReplicatesForReplace = 7,
  modelMatrixType,
  useT = FALSE,
  minmu = if (fitType == "glmGamPoi") 1e-06 else 0.5,
  parallel = FALSE,
  BPPARAM = bpparam()
)
```

Arguments

object	a DESeqDataSet object, see the constructor functions DESeqDataSet, DESeqDataSetFromMatrix, DESeqDataSetFromHTSeqCount.
test	either "Wald" or "LRT", which will then use either Wald significance tests (defined by nbinomWaldTest), or the likelihood ratio test on the difference in deviance between a full and reduced model formula (defined by nbinomLRT)
fitType	either "parametric", "local", "mean", or "glmGamPoi" for the type of fitting of dispersions to the mean intensity. See estimateDispersions for description.
sfType	either "ratio", "poscounts", or "iterate" for the type of size factor estimation. See estimateSizeFactors for description.

betaPrior — whether or not to put a zero-mean normal prior on the non-intercept coefficients See nbinomWaldTest for description of the calculation of the beta prior. In versions >=1.16, the default is set to FALSE, and shrunken LFCs are obtained afterwards using lfcShrink.

full — for test="LRT", the full model formula, which is restricted to the formula in design(object). alternatively, it can be a model matrix constructed by the user. advanced use: specifying a model matrix for full and test="Wald" is possible if betaPrior=FALSE

reduced — for test="LRT", a reduced formula to compare against, i.e., the full formula with the term(s) of interest removed. alternatively, it can be a model matrix constructed by the user

quiet — whether to print messages at each step

minReplicatesForReplace — the minimum number of replicates required in order to use replaceOutliers on a sample. If there are samples with so many replicates, the model will be refit after these replacing outliers, flagged by Cook's distance. Set to Inf in order to never replace outliers. It set to Inf for fitType="glmGamPoi".

modelMatrixType — either "standard" or "expanded", which describe how the model matrix, X of the GLM formula is formed. "standard" is as created by model.matrix using the design formula. "expanded" includes an indicator variable for each level of factors in addition to an intercept. for more information see the Description of nbinomWaldTest. betaPrior must be set to TRUE in order for expanded model matrices to be fit.

useT — logical, passed to nbinomWaldTest, default is FALSE, where Wald statistics are assumed to follow a standard Normal

minmu — lower bound on the estimated count for fitting gene-wise dispersion and for use with nbinomWaldTest and nbinomLRT. If fitType="glmGamPoi", then 1e-6 will be used (as this fitType is optimized for single cell data, where a lower minmu is recommended), otherwise the default value as evaluated on bulk datasets is 0.5

parallel — if FALSE, no parallelization. if TRUE, parallel execution using BiocParallel, see next argument BPPARAM. Two notes on running in parallel using BiocParallel: 1) it is recommended to filter out genes where all samples have low counts before running DESeq2 in parellel: this improves efficiency as otherwise you will be sending data to child processes, though those have little power for detection of differences, and will likely be removed by independent filtering anyway; 2) it may be advantageous to remove large, unneeded objects from your current R environment before calling DESeq, as it is possible that R's internal garbage collection will copy these files while running on worker nodes.

BPPARAM — an optional parameter object passed internally to bplapply when parallel=TRUE. If not specified, the parameters last registered with register will be used.

Details

The differential expression analysis uses a generalized linear model of the form:

$$K_{ij} \sim \text{NB}(\mu_{ij}, \alpha_i)$$

$$\mu_{ij} = s_j q_{ij}$$

$$\log_2(q_{ij}) = x_{j\cdot}\beta_i$$

where counts K_{ij} for gene i, sample j are modeled using a Negative Binomial distribution with fitted mean μ_{ij} and a gene-specific dispersion parameter α_i. The fitted mean is composed of a sample-specific size factor s_j and a parameter q_{ij} proportional to the expected true concentration of fragments for sample j. The coefficients β_i give the log2 fold changes for gene i for each column of the model matrix X. The sample-specific size factors can be replaced by gene-specific normalization factors for each sample using `normalizationFactors`.

For details on the fitting of the log2 fold changes and calculation of p-values, see `nbinomWaldTest` if using `test="Wald"`, or `nbinomLRT` if using `test="LRT"`.

Experiments without replicates do not allow for estimation of the dispersion of counts around the expected value for each group, which is critical for differential expression analysis. Analysis without replicates was deprecated in v1.20 and is no longer supported since v1.22.

The argument `minReplicatesForReplace` is used to decide which samples are eligible for automatic replacement in the case of extreme Cook's distance. By default, `DESeq` will replace outliers if the Cook's distance is large for a sample which has 7 or more replicates (including itself). Outlier replacement is turned off entirely for `fitType="glmGamPoi"`. This replacement is performed by the `replaceOutliers` function. This default behavior helps to prevent filtering genes based on Cook's distance when there are many degrees of freedom. See `results` for more information about filtering using Cook's distance, and the 'Dealing with outliers' section of the vignette. Unlike the behavior of `replaceOutliers`, here original counts are kept in the matrix returned by `counts`, original Cook's distances are kept in `assays(dds)[["cooks"]]`, and the replacement counts used for fitting are kept in `assays(dds)[["replaceCounts"]]`.

Note that if a log2 fold change prior is used (betaPrior=TRUE) then expanded model matrices will be used in fitting. These are described in `nbinomWaldTest` and in the vignette. The `contrast` argument of `results` should be used for generating results tables.

Value

a `DESeqDataSet` object with results stored as metadata columns. These results should accessed by calling the `results` function. By default this will return the log2 fold changes and p-values for the last variable in the design formula. See `results` for how to access results for other variables.

Author(s)

Michael Love

References

Love, M.I., Huber, W., Anders, S. (2014) Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. Genome Biology, 15:550. https://doi.org/10.1186/s13059-014-0550-8

For `fitType="glmGamPoi"`:

Ahlmann-Eltze, C., Huber, W. (2020) glmGamPoi: Fitting Gamma-Poisson Generalized Linear Models on Single Cell Count Data. Bioinformatics. https://doi.org/10.1093/bioinformatics/btaa1009

See Also

`link{results}`, `lfcShrink`, `nbinomWaldTest`, `nbinomLRT`

Examples

```
# see vignette for suggestions on generating
# count tables from RNA-Seq data
cnts <- matrix(rnbinom(n=1000, mu=100, size=1/0.5), ncol=10)
cond <- factor(rep(1:2, each=5))

# object construction
dds <- DESeqDataSetFromMatrix(cnts, DataFrame(cond), ~ cond)

# standard analysis
dds <- DESeq(dds)
res <- results(dds)

# moderated log2 fold changes
resultsNames(dds)
resLFC <- lfcShrink(dds, coef=2, type="apeglm")

# an alternate analysis: likelihood ratio test
ddsLRT <- DESeq(dds, test="LRT", reduced= ~ 1)
resLRT <- results(ddsLRT)
```

DESeqDataSet-class *DESeqDataSet object and constructors*

Description

DESeqDataSet is a subclass of RangedSummarizedExperiment, used to store the input values, intermediate calculations and results of an analysis of differential expression. The DESeqDataSet class enforces non-negative integer values in the "counts" matrix stored as the first element in the assay list. In addition, a formula which specifies the design of the experiment must be provided. The constructor functions create a DESeqDataSet object from various types of input: a RangedSummarizedExperiment, a matrix, count files generated by the python package HTSeq, or a list from the tximport function in the tximport package. See the vignette for examples of construction from different types.

Usage

```
DESeqDataSet(se, design, ignoreRank = FALSE, skipIntegerMode = FALSE)

DESeqDataSetFromMatrix(
  countData,
  colData,
  design,
  tidy = FALSE,
  ignoreRank = FALSE,
  ...
)

DESeqDataSetFromHTSeqCount(
  sampleTable,
  directory = ".",
```

```
  design,
  ignoreRank = FALSE,
  ...
)

DESeqDataSetFromTximport(txi, colData, design, ...)
```

Arguments

se	a RangedSummarizedExperiment with columns of variables indicating sample information in colData, and the counts as the first element in the assays list, which will be renamed "counts". A RangedSummarizedExperiment object can be generated by the function summarizeOverlaps in the GenomicAlignments package.
design	a formula or matrix. the formula expresses how the counts for each gene depend on the variables in colData. Many R formula are valid, including designs with multiple variables, e.g., ~ group + condition, and designs with interactions, e.g., ~ genotype + treatment + genotype:treatment. See results for a variety of designs and how to extract results tables. By default, the functions in this package will use the last variable in the formula for building results tables and plotting. ~ 1 can be used for no design, although users need to remember to switch to another design for differential testing.
ignoreRank	use of this argument is reserved for DEXSeq developers only. Users will immediately encounter an error upon trying to estimate dispersion using a design with a model matrix which is not full rank.
skipIntegerMode	do not convert counts to integer mode (glmGamPoi allows non-integer counts)
countData	for matrix input: a matrix of non-negative integers
colData	for matrix input: a DataFrame or data.frame with at least a single column. Rows of colData correspond to columns of countData
tidy	for matrix input: whether the first column of countData is the rownames for the count matrix
...	arguments provided to SummarizedExperiment including rowRanges and metadata. Note that for Bioconductor 3.1, rowRanges must be a GRanges or GRangesList, with potential metadata columns as a DataFrame accessed and stored with mcols. If a user wants to store metadata columns about the rows of the countData, but does not have GRanges or GRangesList information, first construct the DESeqDataSet without rowRanges and then add the DataFrame with mcols(dds).
sampleTable	for htseq-count: a data.frame with three or more columns. Each row describes one sample. The first column is the sample name, the second column the file name of the count file generated by htseq-count, and the remaining columns are sample metadata which will be stored in colData
directory	for htseq-count: the directory relative to which the filenames are specified. defaults to current directory
txi	for tximport: the simple list output of the tximport function

Details

Note on the error message "assay colnames() must be NULL or equal colData rownames()": this means that the colnames of countData are different than the rownames of colData. Fix this with: colnames(countData) <- NULL

Value

A DESeqDataSet object.

References

See http://www-huber.embl.de/users/anders/HTSeq for htseq-count

Examples

```
countData <- matrix(1:100,ncol=4)
condition <- factor(c("A","A","B","B"))
dds <- DESeqDataSetFromMatrix(countData, DataFrame(condition), ~ condition)
```

DESeqResults-class *DESeqResults object and constructor*

Description

This constructor function would not typically be used by "end users". This simple class indirectly extends the DataFrame class defined in the S4Vectors package to allow other packages to write methods for results objects from the DESeq2 package. It is used by `results` to wrap up the results table.

Usage

```
DESeqResults(DataFrame, priorInfo = list())
```

Arguments

`DataFrame`	a DataFrame of results, standard column names are: baseMean, log2FoldChange, lfcSE, stat, pvalue, padj.
`priorInfo`	a list giving information on the log fold change prior

Value

a DESeqResults object

DESeqTransform-class *DESeqTransform object and constructor*

Description

This constructor function would not typically be used by "end users". This simple class extends the RangedSummarizedExperiment class of the SummarizedExperiment package. It is used by `rlog` and `varianceStabilizingTransformation` to wrap up the results into a class for downstream methods, such as `plotPCA`.

Usage

```
DESeqTransform(SummarizedExperiment)
```

Arguments

SummarizedExperiment	a RangedSummarizedExperiment

Value

a DESeqTransform object

`design,DESeqDataSet-method` *Accessors for the 'design' slot of a DESeqDataSet object.*

Description

The design holds the R `formula` which expresses how the counts depend on the variables in `colData`. See `DESeqDataSet` for details.

Usage

```
## S4 method for signature 'DESeqDataSet'
design(object)

## S4 replacement method for signature 'DESeqDataSet,formula'
design(object) <- value

## S4 replacement method for signature 'DESeqDataSet,matrix'
design(object) <- value
```

Arguments

object	a `DESeqDataSet` object
value	a `formula` used for estimating dispersion and fitting Negative Binomial GLMs

Examples

```
dds <- makeExampleDESeqDataSet(m=4)
design(dds) <- formula(~ 1)
```

dispersionFunction *Accessors for the 'dispersionFunction' slot of a DESeqDataSet object.*

Description

The dispersion function is calculated by estimateDispersions and used by varianceStabilizingTransformation. Parametric dispersion fits store the coefficients of the fit as attributes in this slot.

Usage

```
dispersionFunction(object, ...)

dispersionFunction(object, ...) <- value

## S4 method for signature 'DESeqDataSet'
dispersionFunction(object)

## S4 replacement method for signature 'DESeqDataSet,function'
dispersionFunction(object) <- value
```

Arguments

object	a DESeqDataSet object.
...	additional arguments
value	a function

Details

Using this setter function will also overwrite mcols(object)$dispFit and the estimate of the variance of dispersion residuals.

See Also

estimateDispersions

Examples

```
dds <- makeExampleDESeqDataSet(m=4)
dds <- estimateSizeFactors(dds)
dds <- estimateDispersions(dds)
dispersionFunction(dds)
```

`dispersions` *Accessor functions for the dispersion estimates in a DESeqDataSet object.*

Description

The dispersions for each row of the DESeqDataSet. Generally, these are set by `estimateDispersions`.

Usage

```
dispersions(object, ...)

dispersions(object, ...) <- value

## S4 method for signature 'DESeqDataSet'
dispersions(object)

## S4 replacement method for signature 'DESeqDataSet,numeric'
dispersions(object) <- value
```

Arguments

`object`	a `DESeqDataSet` object.
`...`	additional arguments
`value`	the dispersions to use for the Negative Binomial modeling

Author(s)

Simon Anders

See Also

`estimateDispersions`

`estimateBetaPriorVar` *Steps for estimating the beta prior variance*

Description

These lower-level functions are called within `DESeq` or `nbinomWaldTest`. End users should use those higher-level function instead. NOTE: `estimateBetaPriorVar` returns a numeric vector, not a DESEqDataSet! For advanced users: to use these functions, first run `estimateMLEForBetaPriorVar` and then run `estimateBetaPriorVar`.

Usage

```
estimateBetaPriorVar(
  object,
  betaPriorMethod = c("weighted", "quantile"),
  upperQuantile = 0.05,
  modelMatrix = NULL
)

estimateMLEForBetaPriorVar(
  object,
  maxit = 100,
  useOptim = TRUE,
  useQR = TRUE,
  modelMatrixType = NULL
)
```

Arguments

object	a DESeqDataSet
betaPriorMethod	the method for calculating the beta prior variance, either "quanitle" or "weighted": "quantile" matches a normal distribution using the upper quantile of the finite MLE betas. "weighted" matches a normal distribution using the upper quantile, but weighting by the variance of the MLE betas.
upperQuantile	the upper quantile to be used for the "quantile" or "weighted" method of beta prior variance estimation
modelMatrix	an optional matrix, typically this is set to NULL and created within the function
maxit	as defined in link{nbinomWaldTest}
useOptim	as defined in link{nbinomWaldTest}
useQR	as defined in link{nbinomWaldTest}
modelMatrixType	an optional override for the type which is set internally

Value

for estimateMLEForBetaPriorVar, a DESeqDataSet, with the necessary information stored in order to calculate the prior variance. for estimateBetaPriorVar, the vector of variances for the prior on the betas in the DESeq GLM

---

estimateDispersions,DESeqDataSet-method

Estimate the dispersions for a DESeqDataSet

Description

This function obtains dispersion estimates for Negative Binomial distributed data.

Usage

```
## S4 method for signature 'DESeqDataSet'
estimateDispersions(
  object,
  fitType = c("parametric", "local", "mean", "glmGamPoi"),
  maxit = 100,
  useCR = TRUE,
  weightThreshold = 0.01,
  quiet = FALSE,
  modelMatrix = NULL,
  minmu = if (fitType == "glmGamPoi") 1e-06 else 0.5
)
```

Arguments

`object`	a DESeqDataSet
`fitType`	either "parametric", "local", "mean", or "glmGamPoi" for the type of fitting of dispersions to the mean intensity. • parametric - fit a dispersion-mean relation of the form: $$dispersion = asymptDisp + extraPois/mean$$ via a robust gamma-family GLM. The coefficients `asymptDisp` and `extraPois` are given in the attribute `coefficients` of the `dispersionFunction` of the object. • local - use the locfit package to fit a local regression of log dispersions over log base mean (normal scale means and dispersions are input and output for `dispersionFunction`). The points are weighted by normalized mean count in the local regression. • mean - use the mean of gene-wise dispersion estimates. • glmGamPoi - use the glmGamPoi package to fit the gene-wise dispersion, its trend and calculate the MAP based on the quasi-likelihood framework. The trend is calculated using a local median regression.
`maxit`	control parameter: maximum number of iterations to allow for convergence
`useCR`	whether to use Cox-Reid correction - see McCarthy et al (2012)
`weightThreshold`	threshold for subsetting the design matrix and GLM weights for calculating the Cox-Reid correction
`quiet`	whether to print messages at each step
`modelMatrix`	an optional matrix which will be used for fitting the expected counts. by default, the model matrix is constructed from `design(object)`
`minmu`	lower bound on the estimated count for fitting gene-wise dispersion

Details

Typically the function is called with the idiom:

`dds <- estimateDispersions(dds)`

The fitting proceeds as follows: for each gene, an estimate of the dispersion is found which maximizes the Cox Reid-adjusted profile likelihood (the methods of Cox Reid-adjusted profile likelihood maximization for estimation of dispersion in RNA-Seq data were developed by McCarthy, et

al. (2012), first implemented in the edgeR package in 2010); a trend line capturing the dispersion-mean relationship is fit to the maximum likelihood estimates; a normal prior is determined for the log dispersion estimates centered on the predicted value from the trended fit with variance equal to the difference between the observed variance of the log dispersion estimates and the expected sampling variance; finally maximum a posteriori dispersion estimates are returned. This final dispersion parameter is used in subsequent tests. The final dispersion estimates can be accessed from an object using `dispersions`. The fitted dispersion-mean relationship is also used in `varianceStabilizingTransformation`. All of the intermediate values (gene-wise dispersion estimates, fitted dispersion estimates from the trended fit, etc.) are stored in `mcols(dds)`, with information about these columns in `mcols(mcols(dds))`.

The log normal prior on the dispersion parameter has been proposed by Wu, et al. (2012) and is also implemented in the DSS package.

In DESeq2, the dispersion estimation procedure described above replaces the different methods of dispersion from the previous version of the DESeq package.

Since version 1.29, DESeq2 can call the glmGamPoi package, which can speed up the inference and is optimized for fitting many samles with very small counts (for example single cell RNA-seq data). To call functions from the glmGamPoi package, make sure that it is installed and set `fitType = "glmGamPoi"`. In addition, to the gene estimates, the trend and the MAP, the glmGamPoi package calculates the corresponding quasi-likelihood estimates. Those can be used with the `nbinomLRT()` test to get more precise p-value estimates.

The lower-level functions called by `estimateDispersions` are: `estimateDispersionsGeneEst`, `estimateDispersionsFit`, and `estimateDispersionsMAP`.

Value

The DESeqDataSet passed as parameters, with the dispersion information filled in as metadata columns, accessible via `mcols`, or the final dispersions accessible via `dispersions`.

References

- Simon Anders, Wolfgang Huber: Differential expression analysis for sequence count data. Genome Biology 11 (2010) R106, http://dx.doi.org/10.1186/gb-2010-11-10-r106
- McCarthy, DJ, Chen, Y, Smyth, GK: Differential expression analysis of multifactor RNA-Seq experiments with respect to biological variation. Nucleic Acids Research 40 (2012), 4288-4297, http://dx.doi.org/10.1093/nar/gks042
- Wu, H., Wang, C. & Wu, Z. A new shrinkage estimator for dispersion improves differential expression detection in RNA-seq data. Biostatistics (2012). http://dx.doi.org/10.1093/biostatistics/kxs033
- Ahlmann-Eltze, C., Huber, W. glmGamPoi: Fitting Gamma-Poisson Generalized Linear Models on Single Cell Count Data. Bioinformatics (2020). https://doi.org/10.1093/bioinformatics/btaa1009

Examples

```
dds <- makeExampleDESeqDataSet()
dds <- estimateSizeFactors(dds)
dds <- estimateDispersions(dds)
head(dispersions(dds))
```

`estimateDispersionsGeneEst` *Low-level functions to fit dispersion estimates*

Description

Normal users should instead use `estimateDispersions`. These low-level functions are called by `estimateDispersions`, but are exported and documented for non-standard usage. For instance, it is possible to replace fitted values with a custom fit and continue with the maximum a posteriori dispersion estimation, as demonstrated in the examples below.

Usage

```
estimateDispersionsGeneEst(
  object,
  minDisp = 1e-08,
  kappa_0 = 1,
  dispTol = 1e-06,
  maxit = 100,
  useCR = TRUE,
  weightThreshold = 0.01,
  quiet = FALSE,
  modelMatrix = NULL,
  niter = 1,
  linearMu = NULL,
  minmu = if (type == "glmGamPoi") 1e-06 else 0.5,
  alphaInit = NULL,
  type = c("DESeq2", "glmGamPoi")
)

estimateDispersionsFit(
  object,
  fitType = c("parametric", "local", "mean", "glmGamPoi"),
  minDisp = 1e-08,
  quiet = FALSE
)

estimateDispersionsMAP(
  object,
  outlierSD = 2,
  dispPriorVar,
  minDisp = 1e-08,
  kappa_0 = 1,
  dispTol = 1e-06,
  maxit = 100,
  useCR = TRUE,
  weightThreshold = 0.01,
  modelMatrix = NULL,
  type = c("DESeq2", "glmGamPoi"),
  quiet = FALSE
)
```

```
estimateDispersionsPriorVar(object, minDisp = 1e-08, modelMatrix = NULL)
```

Arguments

object	a DESeqDataSet
minDisp	small value for the minimum dispersion, to allow for calculations in log scale, one order of magnitude above this value is used as a test for inclusion in mean-dispersion fitting
kappa_0	control parameter used in setting the initial proposal in backtracking search, higher kappa_0 results in larger steps
dispTol	control parameter to test for convergence of log dispersion, stop when increase in log posterior is less than dispTol
maxit	control parameter: maximum number of iterations to allow for convergence
useCR	whether to use Cox-Reid correction
weightThreshold	threshold for subsetting the design matrix and GLM weights for calculating the Cox-Reid correction
quiet	whether to print messages at each step
modelMatrix	for advanced use only, a substitute model matrix for gene-wise and MAP dispersion estimation
niter	number of times to iterate between estimation of means and estimation of dispersion
linearMu	estimate the expected counts matrix using a linear model, default is NULL, in which case a lienar model is used if the number of groups defined by the model matrix is equal to the number of columns of the model matrix
minmu	lower bound on the estimated count for fitting gene-wise dispersion
alphaInit	initial guess for the dispersion estimate, alpha
type	can either be "DESeq2" or "glmGamPoi". Specifies if the glmGamPoi package is used to calculate the dispersion. This can be significantly faster if there are many replicates with small counts.
fitType	either "parametric", "local", "mean", or "glmGamPoi" for the type of fitting of dispersions to the mean intensity. See estimateDispersions for description.
outlierSD	the number of standard deviations of log gene-wise estimates above the prior mean (fitted value), above which dispersion estimates will be labelled outliers. Outliers will keep their original value and not be shrunk using the prior.
dispPriorVar	the variance of the normal prior on the log dispersions. If not supplied, this is calculated as the difference between the mean squared residuals of gene-wise estimates to the fitted dispersion and the expected sampling variance of the log dispersion

Value

a DESeqDataSet with gene-wise, fitted, or final MAP dispersion estimates in the metadata columns of the object.

estimateDispersionsPriorVar is called inside of estimateDispersionsMAP and stores the dispersion prior variance as an attribute of dispersionFunction(dds), which can be manually provided to estimateDispersionsMAP for parallel execution.

See Also

estimateDispersions

Examples

```
dds <- makeExampleDESeqDataSet()
dds <- estimateSizeFactors(dds)
dds <- estimateDispersionsGeneEst(dds)
dds <- estimateDispersionsFit(dds)
dds <- estimateDispersionsMAP(dds)
plotDispEsts(dds)

# after having run estimateDispersionsFit()
# the dispersion prior variance over all genes
# can be obtained like so:

dispPriorVar <- estimateDispersionsPriorVar(dds)
```

---

estimateSizeFactors,DESeqDataSet-method

Estimate the size factors for a DESeqDataSet

Description

This function estimates the size factors using the "median ratio method" described by Equation 5 in Anders and Huber (2010). The estimated size factors can be accessed using the accessor function sizeFactors. Alternative library size estimators can also be supplied using the assignment function sizeFactors<-.

Usage

```
## S4 method for signature 'DESeqDataSet'
estimateSizeFactors(
  object,
  type = c("ratio", "poscounts", "iterate"),
  locfunc = stats::median,
  geoMeans,
  controlGenes,
  normMatrix,
  quiet = FALSE
)
```

Arguments

object	a DESeqDataSet
type	Method for estimation: either "ratio", "poscounts", or "iterate". "ratio" uses the standard median ratio method introduced in DESeq. The size factor is the median ratio of the sample over a "pseudosample": for each gene, the geometric mean of all samples. "poscounts" and "iterate" offer alternative estimators, which can be used even when all genes contain a sample with a zero (a problem for the default method, as the geometric mean becomes zero, and the ratio

undefined). The "poscounts" estimator deals with a gene with some zeros, by calculating a modified geometric mean by taking the n-th root of the product of the non-zero counts. This evolved out of use cases with Paul McMurdie's phyloseq package for metagenomic samples. The "iterate" estimator iterates between estimating the dispersion with a design of ~1, and finding a size factor vector by numerically optimizing the likelihood of the ~1 model.

`locfunc` a function to compute a location for a sample. By default, the median is used. However, especially for low counts, the `shorth` function from the genefilter package may give better results.

`geoMeans` by default this is not provided and the geometric means of the counts are calculated within the function. A vector of geometric means from another count matrix can be provided for a "frozen" size factor calculation. The size factors will be scaled to have a geometric mean of 1 when supplying `geoMeans`.

`controlGenes` optional, numeric or logical index vector specifying those genes to use for size factor estimation (e.g. housekeeping or spike-in genes)

`normMatrix` optional, a matrix of normalization factors which do not yet control for library size. Note that this argument should not be used (and will be ignored) if the `dds` object was created using `tximport`. In this case, the information in `assays(dds)[["avgTxLength"]]` is automatically used to create appropriate normalization factors. Providing `normMatrix` will estimate size factors on the count matrix divided by `normMatrix` and store the product of the size factors and `normMatrix` as `normalizationFactors`. It is recommended to divide out the row-wise geometric mean of `normMatrix` so the rows roughly are centered on 1.

`quiet` whether to print messages

Details

Typically, the function is called with the idiom:

```
dds <- estimateSizeFactors(dds)
```

See `DESeq` for a description of the use of size factors in the GLM. One should call this function after `DESeqDataSet` unless size factors are manually specified with `sizeFactors`. Alternatively, gene-specific normalization factors for each sample can be provided using `normalizationFactors` which will always preempt `sizeFactors` in calculations.

Internally, the function calls `estimateSizeFactorsForMatrix`, which provides more details on the calculation.

Value

The DESeqDataSet passed as parameters, with the size factors filled in.

Author(s)

Simon Anders

References

Reference for the median ratio method:

Simon Anders, Wolfgang Huber: Differential expression analysis for sequence count data. Genome Biology 2010, 11:106. http://dx.doi.org/10.1186/gb-2010-11-10-r106

See Also

estimateSizeFactorsForMatrix

Examples

```
dds <- makeExampleDESeqDataSet(n=1000, m=4)
dds <- estimateSizeFactors(dds)
sizeFactors(dds)

dds <- estimateSizeFactors(dds, controlGenes=1:200)

m <- matrix(runif(1000 * 4, .5, 1.5), ncol=4)
dds <- estimateSizeFactors(dds, normMatrix=m)
normalizationFactors(dds)[1:3,]

geoMeans <- exp(rowMeans(log(counts(dds))))
dds <- estimateSizeFactors(dds,geoMeans=geoMeans)
sizeFactors(dds)
```

---

estimateSizeFactorsForMatrix

Low-level function to estimate size factors with robust regression.

Description

Given a matrix or data frame of count data, this function estimates the size factors as follows: Each column is divided by the geometric means of the rows. The median (or, if requested, another location estimator) of these ratios (skipping the genes with a geometric mean of zero) is used as the size factor for this column. Typically, one will not call this function directly, but use estimateSizeFactors.

Usage

```
estimateSizeFactorsForMatrix(
  counts,
  locfunc = stats::median,
  geoMeans,
  controlGenes,
  type = c("ratio", "poscounts")
)
```

Arguments

counts	a matrix or data frame of counts, i.e., non-negative integer values
locfunc	a function to compute a location for a sample. By default, the median is used. However, especially for low counts, the shorth function from genefilter may give better results.
geoMeans	by default this is not provided, and the geometric means of the counts are calculated within the function. A vector of geometric means from another count matrix can be provided for a "frozen" size factor calculation

controlGenes	optional, numeric or logical index vector specifying those genes to use for size factor estimation (e.g. housekeeping or spike-in genes)
type	standard median ratio ("ratio") or where the geometric mean is only calculated over positive counts per row ("poscounts")

Value

a vector with the estimates size factors, one element per column

Author(s)

Simon Anders

See Also

estimateSizeFactors

Examples

```
dds <- makeExampleDESeqDataSet()
estimateSizeFactorsForMatrix(counts(dds))
geoMeans <- exp(rowMeans(log(counts(dds))))
estimateSizeFactorsForMatrix(counts(dds),geoMeans=geoMeans)
```

---

fpkm *FPKM: fragments per kilobase per million mapped fragments*

Description

The following function returns fragment counts normalized per kilobase of feature length per million mapped fragments (by default using a robust estimate of the library size, as in estimateSizeFactors).

Usage

```
fpkm(object, robust = TRUE)
```

Arguments

object	a DESeqDataSet
robust	whether to use size factors to normalize rather than taking the column sums of the raw counts, using the fpm function.

Details

The length of the features (e.g. genes) is calculated one of two ways: (1) If there is a matrix named "avgTxLength" in assays(dds), this will take precedence in the length normalization. This occurs when using the tximport-DESeq2 pipeline. (2) Otherwise, feature length is calculated from the rowRanges of the dds object, if a column basepairs is not present in mcols(dds). The calculated length is the number of basepairs in the union of all GRanges assigned to a given row of object, e.g., the union of all basepairs of exons of a given gene. Note that the second approach over-estimates the

gene length (average transcript length, weighted by abundance is a more appropriate normalization for gene counts), and so the FPKM will be an underestimate of the true value.

Note that, when the read/fragment counting has inter-feature dependencies, a strict normalization would not incorporate the basepairs of a feature which overlap another feature. This inter-feature dependence is not taken into consideration in the internal union basepair calculation.

Value

a matrix which is normalized per kilobase of the union of basepairs in the `GRangesList` or `GRanges` of the mcols(object), and per million of mapped fragments, either using the robust median ratio method (robust=TRUE, default) or using raw counts (robust=FALSE). Defining a column `mcols(object)$basepairs` takes precedence over internal calculation of the kilobases for each row.

See Also

`fpm`

Examples

```
# create a matrix with 1 million counts for the
# 2nd and 3rd column, the 1st and 4th have
# half and double the counts, respectively.
m <- matrix(1e6 * rep(c(.125, .25, .25, .5), each=4),
            ncol=4, dimnames=list(1:4,1:4))
mode(m) <- "integer"
se <- SummarizedExperiment(list(counts=m), colData=DataFrame(sample=1:4))
dds <- DESeqDataSet(se, ~ 1)

# create 4 GRanges with lengths: 1, 1, 2, 2.5 Kb
gr1 <- GRanges("chr1",IRanges(1,1000)) # 1kb
gr2 <- GRanges("chr1",IRanges(c(1,1001),c( 500,1500))) # 1kb
gr3 <- GRanges("chr1",IRanges(c(1,1001),c(1000,2000))) # 2kb
gr4 <- GRanges("chr1",IRanges(c(1,1001),c(200,1300))) # 500bp
rowRanges(dds) <- GRangesList(gr1,gr2,gr3,gr4)

# the raw counts
counts(dds)

# the FPM values
fpm(dds)

# the FPKM values
fpkm(dds)
```

`fpm` *FPM: fragments per million mapped fragments*

Description

Calculates either a robust version (default) or the traditional matrix of fragments/counts per million mapped fragments (FPM/CPM). Note: this function is written very simply and can be easily altered to produce other behavior by examining the source code.

Usage

```
fpm(object, robust = TRUE)
```

Arguments

object	a DESeqDataSet
robust	whether to use size factors to normalize rather than taking the column sums of the raw counts. If TRUE, the size factors and the geometric mean of column sums are multiplied to create a robust library size estimate. Robust normalization is not used if average transcript lengths are present.

Value

a matrix which is normalized per million of mapped fragments, either using the robust median ratio method (robust=TRUE, default) or using raw counts (robust=FALSE).

See Also

fpkm

Examples

```
# generate a dataset with size factors: .5, 1, 1, 2
dds <- makeExampleDESeqDataSet(m = 4, n = 1000,
                               interceptMean=log2(1e3),
                               interceptSD=0,
                               sizeFactors=c(.5,1,1,2),
                               dispMeanRel=function(x) .01)

# add a few rows with very high count
counts(dds)[4:10,] <- 2e5L

# in this robust version, the counts are comparable across samples
round(head(fpm(dds), 3))

# in this column sum version, the counts are still skewed:
# sample1 < sample2 & 3 < sample 4
round(head(fpm(dds, robust=FALSE), 3))

# the column sums of the robust version
# are not equal to 1e6, but the
# column sums of the non-robust version
# are equal to 1e6 by definition

colSums(fpm(dds))/1e6
colSums(fpm(dds, robust=FALSE))/1e6
```

`integrateWithSingleCell` *Integrate bulk DE results with Bioconductor single-cell RNA-seq datasets*

Description

A function that assists with integration of bulk DE results tables with pre-processed scRNA-seq datasets available on Bioconductor, for downstream visualization tasks. The user is prompted to pick a scRNA-seq dataset from a menu. The output of the function is a list with the original results table, bulk gene counts, and the SingleCellExperiment object selected by the user.

Usage

```
integrateWithSingleCell(res, dds, ...)
```

Arguments

`res`	a results table, as produced via `results`
`dds`	a DESeqDataSet with the bulk gene expression data (should contain gene-level counts)
`...`	additional arguments passed to the dataset-accessing function

Details

This function assists the user in choosing a datset from a menu of options that are selected based on the organism of the current dataset. Currently only human and mouse bulk and single-cell RNA-seq datasets are supported, and it is assumed that the bulk DE dataset has GENCODE or Ensembl gene identifiers. Following the selection of the scRNA-seq dataset, visualization can be performed with a package `vizWithSCE`, which can be installed with `install_github("KwameForbes/vizWithSCE")`.

Value

list containing: res, dds, and a SingleCellExperiment as selected by the user

Author(s)

Kwame Forbes

Examples

```
## Not run:
  # involves interactive menu selection...
  dds <- makeExampleDESeqDataSet()
  rownames(dds) <- paste0("ENSG",1:nrow(dds))
  dds <- DESeq(dds)
  res <- results(dds)
  dat <- integrateWithSingleCell(res, dds)

## End(Not run)
```

`lfcShrink` *Shrink log2 fold changes*

Description

Adds shrunken log2 fold changes (LFC) and SE to a results table from `DESeq` run without LFC shrinkage. For consistency with `results`, the column name `lfcSE` is used here although what is returned is a posterior SD. Three shrinkage estimators for LFC are available via `type` (see the vignette for more details on the estimators). The apeglm publication demonstrates that 'apeglm' and 'ashr' outperform the original 'normal' shrinkage estimator.

Usage

```
lfcShrink(
  dds,
  coef,
  contrast,
  res,
  type = c("apeglm", "ashr", "normal"),
  lfcThreshold = 0,
  svalue = FALSE,
  returnList = FALSE,
  format = c("DataFrame", "GRanges", "GRangesList"),
  saveCols = NULL,
  apeAdapt = TRUE,
  apeMethod = "nbinomCR",
  parallel = FALSE,
  BPPARAM = bpparam(),
  quiet = FALSE,
  ...
)
```

Arguments

`dds`	a DESeqDataSet object, after running `DESeq`
`coef`	the name or number of the coefficient (LFC) to shrink, consult `resultsNames(dds)` after running `DESeq(dds)`. note: only `coef` or `contrast` can be specified, not both. `apeglm` requires use of `coef`. For `normal`, one of `coef` or `contrast` must be provided.
`contrast`	see argument description in `results`. only `coef` or `contrast` can be specified, not both.
`res`	a DESeqResults object. Results table produced by the default pipeline, i.e. `DESeq` followed by `results`. If not provided, it will be generated internally using `coef` or `contrast`. For `ashr`, if `res` is provided, then `coef` and `contrast` are ignored.
`type`	`"apeglm"` is the adaptive Student's t prior shrinkage estimator from the 'apeglm' package; `"ashr"` is the adaptive shrinkage estimator from the 'ashr' package, using a fitted mixture of normals prior - see the Stephens (2016) reference below for citation; `"normal"` is the 2014 DESeq2 shrinkage estimator using a Normal prior;

lfcThreshold	a non-negative value which specifies a log2 fold change threshold (as in results). This can be used with any shrinkage type. It will produce new p-values or s-values testing whether the LFC is greater in absolute value than the threshold. The s-values returned in combination with apeglm or ashr provide the probability of FSOS events, "false sign or small", among the tests with equal or smaller s-value than a given gene's s-value, where "small" is specified by lfcThreshold.
svalue	logical, should p-values and adjusted p-values be replaced with s-values when using apeglm or ashr. s-values provide the probability of false signs among the tests with equal or smaller s-value than a given given's s-value. See Stephens (2016) reference on s-values.
returnList	logical, should lfcShrink return a list, where the first element is the results table, and the second element is the output of apeglm or ashr
format	same as defined in results, either "DataFrame", "GRanges", or "GRangesList"
saveCols	same as defined in results, which metadata columns to pass into output
apeAdapt	logical, should apeglm use the MLE estimates of LFC to adapt the prior, or use default or specified prior.control
apeMethod	what method to run apeglm, which can differ in terms of speed
parallel	if FALSE, no parallelization. if TRUE, parallel execution using BiocParallel, see same argument of DESeq parallelization only used with normal or apeglm
BPPARAM	see same argument of DESeq
quiet	whether to print messages
...	arguments passed to apeglm and ashr

Details

See vignette for a comparison of shrinkage estimators on an example dataset. For all shrinkage methods, details on the prior is included in priorInfo(res), including the fitted_g mixture for ashr.

For type="apeglm": Specifying apeglm passes along DESeq2 MLE log2 fold changes and standard errors to the apeglm function in the apeglm package, and re-estimates posterior LFCs for the coefficient specified by coef.

For type="ashr": Specifying ashr passes along DESeq2 MLE log2 fold changes and standard errors to the ash function in the ashr package, with arguments mixcompdist="normal" and method="shrink".

For type="normal": For design as a formula, shrinkage cannot be applied to coefficients in a model with interaction terms. For user-supplied model matrices, shrinkage is only supported via coef. coef will make use of standard model matrices, while contrast will make use of expanded model matrices, and for the latter, a results object should be provided to res. With numeric- or list-style contrasts, it is possible to use lfcShrink, but likely easier to use DESeq with betaPrior=TRUE followed by results, because the numeric or list should reference the coefficients from the expanded model matrix. These coefficients will be printed to console if 'contrast' is used.

Value

a DESeqResults object with the log2FoldChange and lfcSE columns replaced with shrunken LFC and SE. For consistency with results (and similar to the output of bayesglm) the column name lfcSE is used here, although what is returned is a posterior SD. For normal and for apeglm the estimate is the posterior mode, for ashr it is the posterior mean. priorInfo(res) contains information about the shrinkage procedure, relevant to the various methods specified by type.

References

Publications for the following shrinkage estimators:

`type="apeglm"`:

Zhu, A., Ibrahim, J.G., Love, M.I. (2018) Heavy-tailed prior distributions for sequence count data: removing the noise and preserving large differences. Bioinformatics. https://doi.org/10.1093/bioinformatics/bty895

`type="ashr"`:

Stephens, M. (2016) False discovery rates: a new deal. Biostatistics, 18:2. https://doi.org/10.1093/biostatistics/kxw041

`type="normal"`:

Love, M.I., Huber, W., Anders, S. (2014) Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. Genome Biology, 15:550. https://doi.org/10.1186/s13059-014-0550-8

Related work, the `bayesglm` function in the `arm` package:

Gelman, A., Jakulin, A., Pittau, M.G. and Su, Y.-S. (2009). A Weakly Informative Default Prior Distribution For Logistic And Other Regression Models. The Annals of Applied Statistics 2:4. http://www.stat.columbia.edu/~gelman/research/published/priors11.pdf

Examples

```
set.seed(1)
dds <- makeExampleDESeqDataSet(n=500,betaSD=1)
dds <- DESeq(dds)
res <- results(dds)

# these are the coefficients from the model
# we can specify them using 'coef' by name or number below
resultsNames(dds)

res.ape <- lfcShrink(dds=dds, coef=2, type="apeglm")
res.ash <- lfcShrink(dds=dds, coef=2, type="ashr")
res.norm <- lfcShrink(dds=dds, coef=2, type="normal")
```

`makeExampleDESeqDataSet` *Make a simulated DESeqDataSet*

Description

Constructs a simulated dataset of Negative Binomial data from two conditions. By default, there are no fold changes between the two conditions, but this can be adjusted with the `betaSD` argument.

Usage

```
makeExampleDESeqDataSet(
  n = 1000,
  m = 12,
  betaSD = 0,
  interceptMean = 4,
```

```
  interceptSD = 2,
  dispMeanRel = function(x) 4/x + 0.1,
  sizeFactors = rep(1, m)
)
```

Arguments

n	number of rows
m	number of columns
betaSD	the standard deviation for non-intercept betas, i.e. beta ~ N(0,betaSD)
interceptMean	the mean of the intercept betas (log2 scale)
interceptSD	the standard deviation of the intercept betas (log2 scale)
dispMeanRel	a function specifying the relationship of the dispersions on 2^trueIntercept
sizeFactors	multiplicative factors for each sample

Value

a DESeqDataSet with true dispersion, intercept and beta values in the metadata columns. Note that the true betas are provided on the log2 scale.

Examples

```
dds <- makeExampleDESeqDataSet()
dds
```

---

nbinomLRT *Likelihood ratio test (chi-squared test) for GLMs*

Description

This function tests for significance of change in deviance between a full and reduced model which are provided as formula. Fitting uses previously calculated sizeFactors (or normalizationFactors) and dispersion estimates.

Usage

```
nbinomLRT(
  object,
  full = design(object),
  reduced,
  betaTol = 1e-08,
  maxit = 100,
  useOptim = TRUE,
  quiet = FALSE,
  useQR = TRUE,
  minmu = if (type == "glmGamPoi") 1e-06 else 0.5,
  type = c("DESeq2", "glmGamPoi")
)
```

Arguments

object	a DESeqDataSet
full	the full model formula, this should be the formula in design(object). alternatively, can be a matrix
reduced	a reduced formula to compare against, e.g. the full model with a term or terms of interest removed. alternatively, can be a matrix
betaTol	control parameter defining convergence
maxit	the maximum number of iterations to allow for convergence of the coefficient vector
useOptim	whether to use the native optim function on rows which do not converge within maxit
quiet	whether to print messages at each step
useQR	whether to use the QR decomposition on the design matrix X while fitting the GLM
minmu	lower bound on the estimated count while fitting the GLM
type	either "DESeq2" or "glmGamPoi". If type = "DESeq2" a classical likelihood ratio test based on the Chi-squared distribution is conducted. If type = "glmGamPoi" and previously the dispersion has been estimated with glmGamPoi as well, a quasi-likelihood ratio test based on the F-distribution is conducted. It is supposed to be more accurate, because it takes the uncertainty of dispersion estimate into account in the same way that a t-test improves upon a Z-test.

Details

The difference in deviance is compared to a chi-squared distribution with df = (reduced residual degrees of freedom - full residual degrees of freedom). This function is comparable to the nbinomGLMTest of the previous version of DESeq and an alternative to the default nbinomWaldTest.

Value

a DESeqDataSet with new results columns accessible with the results function. The coefficients and standard errors are reported on a log2 scale.

See Also

DESeq, nbinomWaldTest

Examples

```
dds <- makeExampleDESeqDataSet()
dds <- estimateSizeFactors(dds)
dds <- estimateDispersions(dds)
dds <- nbinomLRT(dds, reduced = ~ 1)
res <- results(dds)
```

nbinomWaldTest *Wald test for the GLM coefficients*

Description

This function tests for significance of coefficients in a Negative Binomial GLM, using previously calculated `sizeFactors` (or `normalizationFactors`) and dispersion estimates. See `DESeq` for the GLM formula.

Usage

```
nbinomWaldTest(
  object,
  betaPrior = FALSE,
  betaPriorVar,
  modelMatrix = NULL,
  modelMatrixType,
  betaTol = 1e-08,
  maxit = 100,
  useOptim = TRUE,
  quiet = FALSE,
  useT = FALSE,
  df,
  useQR = TRUE,
  minmu = 0.5
)
```

Arguments

`object`	a DESeqDataSet
`betaPrior`	whether or not to put a zero-mean normal prior on the non-intercept coefficients
`betaPriorVar`	a vector with length equal to the number of model terms including the intercept. betaPriorVar gives the variance of the prior on the sample betas on the log2 scale. if missing (default) this is estimated from the data
`modelMatrix`	an optional matrix, typically this is set to NULL and created within the function
`modelMatrixType`	either "standard" or "expanded", which describe how the model matrix, X of the formula in `DESeq`, is formed. "standard" is as created by `model.matrix` using the design formula. "expanded" includes an indicator variable for each level of factors in addition to an intercept. betaPrior must be set to TRUE in order for expanded model matrices to be fit.
`betaTol`	control parameter defining convergence
`maxit`	the maximum number of iterations to allow for convergence of the coefficient vector
`useOptim`	whether to use the native optim function on rows which do not converge within maxit
`quiet`	whether to print messages at each step

useT	whether to use a t-distribution as a null distribution, for significance testing of the Wald statistics. If FALSE, a standard normal null distribution is used. See next argument df for information about which t is used. If useT=TRUE then further calls to results will make use of mcols(object)$tDegreesFreedom that is stored by nbinomWaldTest.
df	the degrees of freedom for the t-distribution. This can be of length 1 or the number of rows of object. If this is not specified, the degrees of freedom will be set by the number of samples minus the number of columns of the design matrix used for dispersion estimation. If "weights" are included in the assays(object), then the sum of the weights is used in lieu of the number of samples.
useQR	whether to use the QR decomposition on the design matrix X while fitting the GLM
minmu	lower bound on the estimated count while fitting the GLM

Details

The fitting proceeds as follows: standard maximum likelihood estimates for GLM coefficients (synonymous with "beta", "log2 fold change", "effect size") are calculated. Then, optionally, a zero-centered Normal prior distribution (betaPrior) is assumed for the coefficients other than the intercept.

Note that this posterior log2 fold change estimation is now not the default setting for nbinomWaldTest, as the standard workflow for coefficient shrinkage has moved to an additional function link{lfcShrink}.

For calculating Wald test p-values, the coefficients are scaled by their standard errors and then compared to a standard Normal distribution. The results function without any arguments will automatically perform a contrast of the last level of the last variable in the design formula over the first level. The contrast argument of the results function can be used to generate other comparisons.

The Wald test can be replaced with the nbinomLRT for an alternative test of significance.

Notes on the log2 fold change prior:

The variance of the prior distribution for each non-intercept coefficient is calculated using the observed distribution of the maximum likelihood coefficients. The final coefficients are then maximum a posteriori estimates using this prior (Tikhonov/ridge regularization). See below for details on the prior variance and the Methods section of the DESeq2 manuscript for more detail. The use of a prior has little effect on genes with high counts and helps to moderate the large spread in coefficients for genes with low counts.

The prior variance is calculated by matching the 0.05 upper quantile of the observed MLE coefficients to a zero-centered Normal distribution. In a change of methods since the 2014 paper, the weighted upper quantile is calculated using the wtd.quantile function from the Hmisc package (function has been copied into DESeq2 to avoid extra dependencies). The weights are the inverse of the expected variance of log counts, so the inverse of $1/\bar{\mu} + \alpha_{tr}$ using the mean of normalized counts and the trended dispersion fit. The weighting ensures that noisy estimates of log fold changes from small count genes do not overly influence the calculation of the prior variance. See estimateBetaPriorVar. The final prior variance for a factor level is the average of the estimated prior variance over all contrasts of all levels of the factor.

When a log2 fold change prior is used (betaPrior=TRUE), then nbinomWaldTest will by default use expanded model matrices, as described in the modelMatrixType argument, unless this argument is used to override the default behavior. This ensures that log2 fold changes will be independent of the choice of reference level. In this case, the beta prior variance for each factor is calculated as

the average of the mean squared maximum likelihood estimates for each level and every possible contrast.

Value

a DESeqDataSet with results columns accessible with the `results` function. The coefficients and standard errors are reported on a log2 scale.

See Also

`DESeq`, `nbinomLRT`

Examples

```
dds <- makeExampleDESeqDataSet()
dds <- estimateSizeFactors(dds)
dds <- estimateDispersions(dds)
dds <- nbinomWaldTest(dds)
res <- results(dds)
```

---

normalizationFactors *Accessor functions for the normalization factors in a DESeqDataSet object.*

Description

Gene-specific normalization factors for each sample can be provided as a matrix, which will pre-empt `sizeFactors`. In some experiments, counts for each sample have varying dependence on covariates, e.g. on GC-content for sequencing data run on different days, and in this case it makes sense to provide gene-specific factors for each sample rather than a single size factor.

Usage

```
normalizationFactors(object, ...)

normalizationFactors(object, ...) <- value

## S4 method for signature 'DESeqDataSet'
normalizationFactors(object)

## S4 replacement method for signature 'DESeqDataSet,matrix'
normalizationFactors(object) <- value
```

Arguments

`object`	a `DESeqDataSet` object.
`...`	additional arguments
`value`	the matrix of normalization factors

Details

Normalization factors alter the model of DESeq in the following way, for counts K_{ij} and normalization factors NF_{ij} for gene i and sample j:

$$K_{ij} \sim \text{NB}(\mu_{ij}, \alpha_i)$$
$$\mu_{ij} = NF_{ij} q_{ij}$$

Note

Normalization factors are on the scale of the counts (similar to sizeFactors) and unlike offsets, which are typically on the scale of the predictors (in this case, log counts). Normalization factors should include library size normalization. They should have row-wise geometric mean near 1, as is the case with size factors, such that the mean of normalized counts is close to the mean of unnormalized counts. See example code below.

Examples

```
dds <- makeExampleDESeqDataSet(n=100, m=4)

normFactors <- matrix(runif(nrow(dds)*ncol(dds),0.5,1.5),
                      ncol=ncol(dds),nrow=nrow(dds),
                      dimnames=list(1:nrow(dds),1:ncol(dds)))

# the normalization factors matrix should not have 0's in it
# it should have geometric mean near 1 for each row
normFactors <- normFactors / exp(rowMeans(log(normFactors)))
normalizationFactors(dds) <- normFactors

dds <- DESeq(dds)
```

---

normalizeGeneLength *Normalize for gene length*

Description

Normalize for gene length using the output of transcript abundance estimators

Usage

```
normalizeGeneLength(...)
```

Arguments

...	...

Details

This function is deprecated and moved to a new general purpose package, tximport, which will be added to Bioconductor.

normTransform *Normalized counts transformation*

Description

A simple function for creating a DESeqTransform object after applying: f(count(dds,normalized=TRUE) + pc).

Usage

```
normTransform(object, f = log2, pc = 1)
```

Arguments

object	a DESeqDataSet object
f	a function to apply to normalized counts
pc	a pseudocount to add to normalized counts

See Also

varianceStabilizingTransformation, rlog

plotCounts *Plot of normalized counts for a single gene*

Description

Normalized counts plus a pseudocount of 0.5 are shown by default.

Usage

```
plotCounts(
  dds,
  gene,
  intgroup = "condition",
  normalized = TRUE,
  transform = TRUE,
  main,
  xlab = "group",
  returnData = FALSE,
  replaced = FALSE,
  pc,
  ...
)
```

Arguments

dds	a DESeqDataSet
gene	a character, specifying the name of the gene to plot
intgroup	interesting groups: a character vector of names in colData(x) to use for grouping. Must be factor variables. If you want to plot counts over numeric, choose returnData=TRUE
normalized	whether the counts should be normalized by size factor (default is TRUE)
transform	whether to have log scale y-axis or not. defaults to TRUE
main	as in 'plot'
xlab	as in 'plot'
returnData	should the function only return the data.frame of counts and covariates for custom plotting (default is FALSE)
replaced	use the outlier-replaced counts if they exist
pc	pseudocount for log transform
...	arguments passed to plot

Examples

```
dds <- makeExampleDESeqDataSet()
plotCounts(dds, "gene1")
```

---

plotDispEsts *Plot dispersion estimates*

Description

A simple helper function that plots the per-gene dispersion estimates together with the fitted mean-dispersion relationship.

Usage

```
## S4 method for signature 'DESeqDataSet'
plotDispEsts(
  object,
  ymin,
  CV = FALSE,
  genecol = "black",
  fitcol = "red",
  finalcol = "dodgerblue",
  legend = TRUE,
  xlab,
  ylab,
  log = "xy",
  cex = 0.45,
  ...
)
```

Arguments

object	a DESeqDataSet, with dispersions estimated
ymin	the lower bound for points on the plot, points beyond this are drawn as triangles at ymin
CV	logical, whether to plot the asymptotic or biological coefficient of variation (the square root of dispersion) on the y-axis. As the mean grows to infinity, the square root of dispersion gives the coefficient of variation for the counts. Default is FALSE, plotting dispersion.
genecol	the color for gene-wise dispersion estimates
fitcol	the color of the fitted estimates
finalcol	the color of the final estimates used for testing
legend	logical, whether to draw a legend
xlab	xlab
ylab	ylab
log	log
cex	cex
...	further arguments to plot

Author(s)

Simon Anders

Examples

```
dds <- makeExampleDESeqDataSet()
dds <- estimateSizeFactors(dds)
dds <- estimateDispersions(dds)
plotDispEsts(dds)
```

---

plotMA *MA-plot from base means and log fold changes*

Description

A simple helper function that makes a so-called "MA-plot", i.e. a scatter plot of log2 fold changes (on the y-axis) versus the mean of normalized counts (on the x-axis).

Usage

```
## S4 method for signature 'DESeqDataSet'
plotMA(
  object,
  alpha = 0.1,
  main = "",
  xlab = "mean of normalized counts",
  ylim,
  colNonSig = "gray60",
```

```
  colSig = "blue",
  colLine = "grey40",
  returnData = FALSE,
  MLE = FALSE,
  ...
)

## S4 method for signature 'DESeqResults'
plotMA(
  object,
  alpha,
  main = "",
  xlab = "mean of normalized counts",
  ylim,
  colNonSig = "gray60",
  colSig = "blue",
  colLine = "grey40",
  returnData = FALSE,
  MLE = FALSE,
  ...
)
```

Arguments

object	a DESeqResults object produced by results; or a DESeqDataSet processed by DESeq, or the individual functions nbinomWaldTest or nbinomLRT
alpha	the significance level for thresholding adjusted p-values
main	optional title for the plot
xlab	optional defaults to "mean of normalized counts"
ylim	optional y limits
colNonSig	color to use for non-significant data points
colSig	color to use for significant data points
colLine	color to use for the horizontal (y=0) line
returnData	logical, whether to return the data.frame used for plotting
MLE	if betaPrior=TRUE was used, whether to plot the MLE (unshrunken estimates), defaults to FALSE. Requires that results was run with addMLE=TRUE. Note that the MLE will be plotted regardless of this argument, if DESeq() was run with betaPrior=FALSE. See lfcShrink for examples on how to plot shrunken log2 fold changes.
...	further arguments passed to plotMA if object is DESeqResults or to results if object is DESeqDataSet

Details

This function is essentially two lines of code: building a data.frame and passing this to the plotMA method for data.frame, now copied from the geneplotter package. The code was modified in version 1.28 to change from red to blue points for better visibility for users with color-blindness. The original plots can still be made via the use of returnData=TRUE and passing the resulting data.frame directly to geneplotter::plotMA. The code of this function can be seen with: getMethod("plotMA","DESeqDataSet") If the object contains a column svalue then these will be used for coloring the points (with a default alpha=0.005).

Author(s)

Michael Love

Examples

```
dds <- makeExampleDESeqDataSet()
dds <- DESeq(dds)
plotMA(dds)
res <- results(dds)
plotMA(res)
```

---

plotPCA *Sample PCA plot for transformed data*

Description

This plot helps to check for batch effects and the like.

Usage

```
## S4 method for signature 'DESeqTransform'
plotPCA(
  object,
  intgroup = "condition",
  ntop = 500,
  returnData = FALSE,
  pcsToUse = 1:2
)
```

Arguments

object	a DESeqTransform object, with data in assay(x), produced for example by either rlog or varianceStabilizingTransformation.
intgroup	interesting groups: a character vector of names in colData(x) to use for grouping
ntop	number of top genes to use for principal components, selected by highest row variance
returnData	should the function only return the data.frame of PC1 and PC2 with intgroup covariates for custom plotting (default is FALSE)
pcsToUse	numeric of length 2, which PCs to plot

Value

An object created by ggplot, which can be assigned and further customized.

Note

See the vignette for an example of variance stabilization and PCA plots. Note that the source code of plotPCA is very simple. The source can be found by typing DESeq2:::plotPCA.DESeqTransform or getMethod("plotPCA","DESeqTransform"), or browsed on github at https://github.com/mikelove/DESeq2/blob/master/R/plots.R Users should find it easy to customize this function.

Author(s)

Wolfgang Huber

Examples

```
# using rlog transformed data:
dds <- makeExampleDESeqDataSet(betaSD=1)
vsd <- vst(dds, nsub=500)
plotPCA(vsd)

# also possible to perform custom transformation:
dds <- estimateSizeFactors(dds)
# shifted log of normalized counts
se <- SummarizedExperiment(log2(counts(dds, normalized=TRUE) + 1),
                           colData=colData(dds))
# the call to DESeqTransform() is needed to
# trigger our plotPCA method.
plotPCA( DESeqTransform( se ) )
```

`plotSparsity` *Sparsity plot*

Description

A simple plot of the concentration of counts in a single sample over the sum of counts per gene. Not technically the same as "sparsity", but this plot is useful diagnostic for datasets which might not fit a negative binomial assumption: genes with many zeros and individual very large counts are difficult to model with the negative binomial distribution.

Usage

```
plotSparsity(x, normalized = TRUE, ...)
```

Arguments

`x`	a matrix or DESeqDataSet
`normalized`	whether to normalize the counts from a DESeqDataSEt
`...`	passed to `plot`

Examples

```
dds <- makeExampleDESeqDataSet(n=1000,m=4,dispMeanRel=function(x) .5)
dds <- estimateSizeFactors(dds)
plotSparsity(dds)
```

priorInfo *Accessors for the 'priorInfo' slot of a DESeqResults object.*

Description

The priorInfo slot contains details about the prior on log fold changes

Usage

```
priorInfo(object, ...)

priorInfo(object, ...) <- value

## S4 method for signature 'DESeqResults'
priorInfo(object)

## S4 replacement method for signature 'DESeqResults,list'
priorInfo(object) <- value
```

Arguments

object	a DESeqResults object
...	additional arguments
value	a list

replaceOutliers *Replace outliers with trimmed mean*

Description

Note that this function is called within DESeq, so is not necessary to call on top of a DESeq call. See the minReplicatesForReplace argument documented in link{DESeq}.

Usage

```
replaceOutliers(
  object,
  trim = 0.2,
  cooksCutoff,
  minReplicates = 7,
  whichSamples
)

replaceOutliersWithTrimmedMean(
  object,
  trim = 0.2,
  cooksCutoff,
  minReplicates = 7,
  whichSamples
)
```

Arguments

object	a DESeqDataSet object, which has already been processed by either DESeq, nbinomWaldTest or nbinomLRT, and therefore contains a matrix contained in assays(dds)[["cooks"]]. These are the Cook's distances which will be used to define outlier counts.
trim	the fraction (0 to 0.5) of observations to be trimmed from each end of the normalized counts for a gene before the mean is computed
cooksCutoff	the threshold for defining an outlier to be replaced. Defaults to the .99 quantile of the F(p, m - p) distribution, where p is the number of parameters and m is the number of samples.
minReplicates	the minimum number of replicate samples necessary to consider a sample eligible for replacement (including itself). Outlier counts will not be replaced if the sample is in a cell which has less than minReplicates replicates.
whichSamples	optional, a numeric or logical index to specify which samples should have outliers replaced. if missing, this is determined using minReplicates.

Details

This function replaces outlier counts flagged by extreme Cook's distances, as calculated by DESeq, nbinomWaldTest or nbinomLRT, with values predicted by the trimmed mean over all samples (and adjusted by size factor or normalization factor). This function replaces the counts in the matrix returned by counts(dds) and the Cook's distances in assays(dds)[["cooks"]]. Original counts are preserved in assays(dds)[["originalCounts"]].

The DESeq function calculates a diagnostic measure called Cook's distance for every gene and every sample. The results function then sets the p-values to NA for genes which contain an outlying count as defined by a Cook's distance above a threshold. With many degrees of freedom, i.e. many more samples than number of parameters to be estimated– it might be undesirable to remove entire genes from the analysis just because their data include a single count outlier. An alternate strategy is to replace the outlier counts with the trimmed mean over all samples, adjusted by the size factor or normalization factor for that sample. The following simple function performs this replacement for the user, for samples which have at least minReplicates number of replicates (including that sample). For more information on Cook's distance, please see the two sections of the vignette: 'Dealing with count outliers' and 'Count outlier detection'.

Value

a DESeqDataSet with replaced counts in the slot returned by counts and the original counts preserved in assays(dds)[["originalCounts"]]

See Also

DESeq

---

results *Extract results from a DESeq analysis*

Description

`results` extracts a result table from a DESeq analysis giving base means across samples, log2 fold changes, standard errors, test statistics, p-values and adjusted p-values; `resultsNames` returns the names of the estimated effects (coefficents) of the model; `removeResults` returns a `DESeqDataSet` object with results columns removed.

Usage

```
results(
  object,
  contrast,
  name,
  lfcThreshold = 0,
 altHypothesis = c("greaterAbs", "lessAbs", "greater", "less", "greaterAbs2014"),
  listValues = c(1, -1),
  cooksCutoff,
  independentFiltering = TRUE,
  alpha = 0.1,
  filter,
  theta,
  pAdjustMethod = "BH",
  filterFun,
  format = c("DataFrame", "GRanges", "GRangesList"),
  saveCols = NULL,
  test = c("Wald", "LRT"),
  addMLE = FALSE,
  tidy = FALSE,
  parallel = FALSE,
  BPPARAM = bpparam(),
  minmu = 0.5
)

resultsNames(object)

removeResults(object)
```

Arguments

`object`	a DESeqDataSet, on which one of the following functions has already been called: `DESeq`, `nbinomWaldTest`, or `nbinomLRT`
`contrast`	this argument specifies what comparison to extract from the `object` to build a results table. one of either: • a character vector with exactly three elements: the name of a factor in the design formula, the name of the numerator level for the fold change, and the name of the denominator level for the fold change (simplest case) • a list of 2 character vectors: the names of the fold changes for the numerator, and the names of the fold changes for the denominator. these names should be elements of `resultsNames(object)`. if the list is length 1, a second element is added which is the empty character vector, `character()`. (more general case, can be to combine interaction terms and main effects) • a numeric contrast vector with one element for each element in `resultsNames(object)` (most general case)

If specified, the name argument is ignored.

name — the name of the individual effect (coefficient) for building a results table. Use this argument rather than contrast for continuous variables, individual effects or for individual interaction terms. The value provided to name must be an element of resultsNames(object).

lfcThreshold — a non-negative value which specifies a log2 fold change threshold. The default value is 0, corresponding to a test that the log2 fold changes are equal to zero. The user can specify the alternative hypothesis using the altHypothesis argument, which defaults to testing for log2 fold changes greater in absolute value than a given threshold. If lfcThreshold is specified, the results are for Wald tests, and LRT p-values will be overwritten.

altHypothesis — character which specifies the alternative hypothesis, i.e. those values of log2 fold change which the user is interested in finding. The complement of this set of values is the null hypothesis which will be tested. If the log2 fold change specified by name or by contrast is written as β, then the possible values for altHypothesis represent the following alternate hypotheses:

- greaterAbs: $|\beta| >$ lfcThreshold, and p-values are two-tailed
- lessAbs: $|\beta| <$ lfcThreshold, p-values are the maximum of the upper and lower tests. The Wald statistic given is positive, an SE-scaled distance from the closest boundary
- greater: $\beta >$ lfcThreshold
- less: $\beta < -$lfcThreshold
- greaterAbs2014: older implementation of greaterAbs from 2014, less power

listValues — only used if a list is provided to contrast: a numeric of length two: the log2 fold changes in the list are multiplied by these values. the first number should be positive and the second negative. by default this is c(1,-1)

cooksCutoff — theshold on Cook's distance, such that if one or more samples for a row have a distance higher, the p-value for the row is set to NA. The default cutoff is the .99 quantile of the F(p, m-p) distribution, where p is the number of coefficients being fitted and m is the number of samples. Set to Inf or FALSE to disable the resetting of p-values to NA. Note: this test excludes the Cook's distance of samples belonging to experimental groups with only 2 samples.

independentFiltering — logical, whether independent filtering should be applied automatically

alpha — the significance cutoff used for optimizing the independent filtering (by default 0.1). If the adjusted p-value cutoff (FDR) will be a value other than 0.1, alpha should be set to that value.

filter — the vector of filter statistics over which the independent filtering will be optimized. By default the mean of normalized counts is used.

theta — the quantiles at which to assess the number of rejections from independent filtering

pAdjustMethod — the method to use for adjusting p-values, see ?p.adjust

filterFun — an optional custom function for performing independent filtering and p-value adjustment, with arguments res (a DESeqResults object), filter (the quantity for filtering tests), alpha (the target FDR), pAdjustMethod. This function should return a DESeqResults object with a padj column.

format — character, either "DataFrame", "GRanges", or "GRangesList", whether the results should be printed as a DESeqResults DataFrame, or if the results DataFrame

should be attached as metadata columns to the GRanges or GRangesList rowRanges of the DESeqDataSet. If the rowRanges is a GRangesList, and GRanges is requested, the range of each gene will be returned

saveCols — character or numeric vector, the columns of mcols(object) to pass into the results output

test — this is automatically detected internally if not provided. the one exception is after nbinomLRT has been run, test="Wald" will generate Wald statistics and Wald test p-values.

addMLE — if betaPrior=TRUE was used (non-default), this logical argument specifies if the "unshrunken" maximum likelihood estimates (MLE) of log2 fold change should be added as a column to the results table (default is FALSE). This argument is preserved for backward compatability, as now betaPrior=FALSE by default and the recommended pipeline is to generate shrunken MAP estimates using lfcShrink. This argument functionality is only implemented for contrast specified as three element character vectors.

tidy — whether to output the results table with rownames as a first column 'row'. the table will also be coerced to data.frame

parallel — if FALSE, no parallelization. if TRUE, parallel execution using BiocParallel, see next argument BPPARAM

BPPARAM — an optional parameter object passed internally to bplapply when parallel=TRUE. If not specified, the parameters last registered with register will be used.

minmu — lower bound on the estimated count (used when calculating contrasts)

Details

The results table when printed will provide the information about the comparison, e.g. "log2 fold change (MAP): condition treated vs untreated", meaning that the estimates are of log2(treated / untreated), as would be returned by contrast=c("condition","treated","untreated"). Multiple results can be returned for analyses beyond a simple two group comparison, so results takes arguments contrast and name to help the user pick out the comparisons of interest for printing a results table. The use of the contrast argument is recommended for exact specification of the levels which should be compared and their order.

If results is run without specifying contrast or name, it will return the comparison of the last level of the last variable in the design formula over the first level of this variable. For example, for a simple two-group comparison, this would return the log2 fold changes of the second group over the first group (the reference level). Please see examples below and in the vignette.

The argument contrast can be used to generate results tables for any comparison of interest, for example, the log2 fold change between two levels of a factor, and its usage is described below. It can also accomodate more complicated numeric comparisons. Note that contrast will set to 0 the estimated LFC in a comparison of two groups, where all of the counts in the two groups are equal to 0 (while other groups have positive counts), while name will not automatically set these LFC to 0. The test statistic used for a contrast is:

$$c^t \beta / \sqrt{c^t \Sigma c}$$

The argument name can be used to generate results tables for individual effects, which must be individual elements of resultsNames(object). These individual effects could represent continuous covariates, effects for individual levels, or individual interaction effects.

Information on the comparison which was used to build the results table, and the statistical test which was used for p-values (Wald test or likelihood ratio test) is stored within the object returned

by results. This information is in the metadata columns of the results table, which is accessible by calling mcols on the DESeqResults object returned by results.

On p-values:

By default, independent filtering is performed to select a set of genes for multiple test correction which maximizes the number of adjusted p-values less than a given critical value alpha (by default 0.1). See the reference in this man page for details on independent filtering. The filter used for maximizing the number of rejections is the mean of normalized counts for all samples in the dataset. Several arguments from the filtered_p function of the genefilter package (used within the results function) are provided here to control the independent filtering behavior. (Note filtered_p R code is now copied into DESeq2 package to avoid gfortran requirements.) In DESeq2 version >= 1.10, the threshold that is chosen is the lowest quantile of the filter for which the number of rejections is close to the peak of a curve fit to the number of rejections over the filter quantiles. 'Close to' is defined as within 1 residual standard deviation. The adjusted p-values for the genes which do not pass the filter threshold are set to NA.

By default, results assigns a p-value of NA to genes containing count outliers, as identified using Cook's distance. See the cooksCutoff argument for control of this behavior. Cook's distances for each sample are accessible as a matrix "cooks" stored in the assays() list. This measure is useful for identifying rows where the observed counts might not fit to a Negative Binomial distribution.

For analyses using the likelihood ratio test (using nbinomLRT), the p-values are determined solely by the difference in deviance between the full and reduced model formula. A single log2 fold change is printed in the results table for consistency with other results table outputs, however the test statistic and p-values may nevertheless involve the testing of one or more log2 fold changes. Which log2 fold change is printed in the results table can be controlled using the name argument, or by default this will be the estimated coefficient for the last element of resultsNames(object).

If useT=TRUE was specified when running DESeq or nbinomWaldTest, then the p-value generated by results will also make use of the t distribution for the Wald statistic, using the degrees of freedom in mcols(object)$tDegreesFreedom.

Value

For results: a DESeqResults object, which is a simple subclass of DataFrame. This object contains the results columns: baseMean, log2FoldChange, lfcSE, stat, pvalue and padj, and also includes metadata columns of variable information. The lfcSE gives the standard error of the log2FoldChange. For the Wald test, stat is the Wald statistic: the log2FoldChange divided by lfcSE, which is compared to a standard Normal distribution to generate a two-tailed pvalue. For the likelihood ratio test (LRT), stat is the difference in deviance between the reduced model and the full model, which is compared to a chi-squared distribution to generate a pvalue.

For resultsNames: the names of the columns available as results, usually a combination of the variable name and a level

For removeResults: the original DESeqDataSet with results metadata columns removed

References

Richard Bourgon, Robert Gentleman, Wolfgang Huber: Independent filtering increases detection power for high-throughput experiments. PNAS (2010), http://dx.doi.org/10.1073/pnas.0914005107

See Also

DESeq, lfcShrink

Examples

```
## Example 1: two-group comparison

dds <- makeExampleDESeqDataSet(m=4)

dds <- DESeq(dds)
res <- results(dds, contrast=c("condition","B","A"))

# with more than two groups, the call would look similar, e.g.:
# results(dds, contrast=c("condition","C","A"))
# etc.

## Example 2: two conditions, two genotypes, with an interaction term

dds <- makeExampleDESeqDataSet(n=100,m=12)
dds$genotype <- factor(rep(rep(c("I","II"),each=3),2))

design(dds) <- ~ genotype + condition + genotype:condition
dds <- DESeq(dds)
resultsNames(dds)

# the condition effect for genotype I (the main effect)
results(dds, contrast=c("condition","B","A"))

# the condition effect for genotype II
# this is, by definition, the main effect *plus* the interaction term
# (the extra condition effect in genotype II compared to genotype I).
results(dds, list( c("condition_B_vs_A","genotypeII.conditionB") ))

# the interaction term, answering: is the condition effect *different* across genotypes?
results(dds, name="genotypeII.conditionB")

## Example 3: two conditions, three genotypes

# ~~~ Using interaction terms ~~~

dds <- makeExampleDESeqDataSet(n=100,m=18)
dds$genotype <- factor(rep(rep(c("I","II","III"),each=3),2))
design(dds) <- ~ genotype + condition + genotype:condition
dds <- DESeq(dds)
resultsNames(dds)

# the condition effect for genotype I (the main effect)
results(dds, contrast=c("condition","B","A"))

# the condition effect for genotype III.
# this is the main effect *plus* the interaction term
# (the extra condition effect in genotype III compared to genotype I).
results(dds, contrast=list( c("condition_B_vs_A","genotypeIII.conditionB") ))

# the interaction term for condition effect in genotype III vs genotype I.
# this tests if the condition effect is different in III compared to I
results(dds, name="genotypeIII.conditionB")

# the interaction term for condition effect in genotype III vs genotype II.
# this tests if the condition effect is different in III compared to II
```

```
results(dds, contrast=list("genotypeIII.conditionB", "genotypeII.conditionB"))

# Note that a likelihood ratio could be used to test if there are any
# differences in the condition effect between the three genotypes.

# ~~~ Using a grouping variable ~~~

# This is a useful construction when users just want to compare
# specific groups which are combinations of variables.

dds$group <- factor(paste0(dds$genotype, dds$condition))
design(dds) <- ~ group
dds <- DESeq(dds)
resultsNames(dds)

# the condition effect for genotypeIII
results(dds, contrast=c("group", "IIIB", "IIIA"))
```

`rlog` *Apply a 'regularized log' transformation*

Description

This function transforms the count data to the log2 scale in a way which minimizes differences between samples for rows with small counts, and which normalizes with respect to library size. The rlog transformation produces a similar variance stabilizing effect as `varianceStabilizingTransformation`, though `rlog` is more robust in the case when the size factors vary widely. The transformation is useful when checking for outliers or as input for machine learning techniques such as clustering or linear discriminant analysis. `rlog` takes as input a `DESeqDataSet` and returns a `RangedSummarizedExperiment` object.

Usage

```
rlog(object, blind = TRUE, intercept, betaPriorVar, fitType = "parametric")

rlogTransformation(
  object,
  blind = TRUE,
  intercept,
  betaPriorVar,
  fitType = "parametric"
)
```

Arguments

`object`	a DESeqDataSet, or matrix of counts
`blind`	logical, whether to blind the transformation to the experimental design. blind=TRUE should be used for comparing samples in an manner unbiased by prior information on samples, for example to perform sample QA (quality assurance). blind=FALSE should be used for transforming data for downstream analysis, where the full use of the design information should be made. blind=FALSE will

skip re-estimation of the dispersion trend, if this has already been calculated. If many of genes have large differences in counts due to the experimental design, it is important to set blind=FALSE for downstream analysis.

`intercept` by default, this is not provided and calculated automatically. if provided, this should be a vector as long as the number of rows of object, which is log2 of the mean normalized counts from a previous dataset. this will enforce the intercept for the GLM, allowing for a "frozen" rlog transformation based on a previous dataset. You will also need to provide `mcols(object)$dispFit`.

`betaPriorVar` a single value, the variance of the prior on the sample betas, which if missing is estimated from the data

`fitType` in case dispersions have not yet been estimated for `object`, this parameter is passed on to `estimateDispersions` (options described there).

Details

Note that neither rlog transformation nor the VST are used by the differential expression estimation in `DESeq`, which always occurs on the raw count data, through generalized linear modeling which incorporates knowledge of the variance-mean dependence. The rlog transformation and VST are offered as separate functionality which can be used for visualization, clustering or other machine learning tasks. See the transformation section of the vignette for more details, including a statement on timing. If `rlog` is run on data with number of samples in [30-49] it will print a message that it may take a few minutes, if the number of samples is 50 or larger, it will print a message that it may take a "long time", and in both cases, it will mention that the `vst` is a much faster transformation.

The transformation does not require that one has already estimated size factors and dispersions.

The regularization is on the log fold changes of the count for each sample over an intercept, for each gene. As nearby count values for low counts genes are almost as likely as the observed count, the rlog shrinkage is greater for low counts. For high counts, the rlog shrinkage has a much weaker effect. The fitted dispersions are used rather than the MAP dispersions (so similar to the `varianceStabilizingTransformation`).

The prior variance for the shrinkag of log fold changes is calculated as follows: a matrix is constructed of the logarithm of the counts plus a pseudocount of 0.5, the log of the row means is then subtracted, leaving an estimate of the log fold changes per sample over the fitted value using only an intercept. The prior variance is then calculated by matching the upper quantiles of the observed log fold change estimates with an upper quantile of the normal distribution. A GLM fit is then calculated using this prior. It is also possible to supply the variance of the prior. See the vignette for an example of the use and a comparison with `varianceStabilizingTransformation`.

The transformed values, rlog(K), are equal to $rlog(K_{ij}) = \log_2(q_{ij}) = \beta_{i0} + \beta_{ij}$, with formula terms defined in `DESeq`.

The parameters of the rlog transformation from a previous dataset can be frozen and reapplied to new samples. See the 'Data quality assessment' section of the vignette for strategies to see if new samples are sufficiently similar to previous datasets. The frozen rlog is accomplished by saving the dispersion function, beta prior variance and the intercept from a previous dataset, and running `rlog` with 'blind' set to FALSE (see example below).

Value

a `DESeqTransform` if a `DESeqDataSet` was provided, or a matrix if a count matrix was provided as input. Note that for `DESeqTransform` output, the matrix of transformed values is stored in `assay(rld)`. To avoid returning matrices with NA values, in the case of a row of all zeros, the rlog transformation returns zeros (essentially adding a pseudocount of 1 only to these rows).

References

Reference for regularized logarithm (rlog):

Michael I Love, Wolfgang Huber, Simon Anders: Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. Genome Biology 2014, 15:550. http://dx.doi.org/10.1186/s13059-014-0550-8

See Also

plotPCA, varianceStabilizingTransformation, normTransform

Examples

```
dds <- makeExampleDESeqDataSet(m=6,betaSD=1)
rld <- rlog(dds)
dists <- dist(t(assay(rld)))
# plot(hclust(dists))
```

---

show,DESeqResults-method

Show method for DESeqResults objects

Description

Prints out the information from the metadata columns of the results object regarding the log2 fold changes and p-values, then shows the DataFrame using the standard method.

Usage

```
## S4 method for signature 'DESeqResults'
show(object)
```

Arguments

object	a DESeqResults object

Author(s)

Michael Love

sizeFactors,DESeqDataSet-method

Accessor functions for the 'sizeFactors' information in a DESeqDataSet object.

Description

The sizeFactors vector assigns to each column of the count matrix a value, the size factor, such that count values in the columns can be brought to a common scale by dividing by the corresponding size factor (as performed by `counts(dds, normalized=TRUE)`). See `DESeq` for a description of the use of size factors. If gene-specific normalization is desired for each sample, use `normalizationFactors`.

Usage

```
## S4 method for signature 'DESeqDataSet'
sizeFactors(object)

## S4 replacement method for signature 'DESeqDataSet,numeric'
sizeFactors(object) <- value
```

Arguments

`object`	a `DESeqDataSet` object.
`value`	a numeric vector, one size factor for each column in the count data.

Author(s)

Simon Anders

See Also

`estimateSizeFactors`

summary,DESeqResults-method

Summarize DESeq results

Description

Print a summary of the results from a DESeq analysis.

Usage

```
## S4 method for signature 'DESeqResults'
summary(object, alpha, ...)
```

Arguments

object	a DESeqResults object
alpha	the adjusted p-value cutoff. If not set, this defaults to the alpha argument which was used in results to set the target FDR for independent filtering, or if independent filtering was not performed, to 0.1.
...	additional arguments

Author(s)

Michael Love

Examples

```
dds <- makeExampleDESeqDataSet(m=4)
dds <- DESeq(dds)
res <- results(dds)
summary(res)
```

---

unmix *Unmix samples using loss in a variance stabilized space*

Description

Unmixes samples in x according to pure components, using numerical optimization. The components in pure are added on the scale of gene expression (either normalized counts, or TPMs). The loss function when comparing fitted expression to the samples in x occurs in a variance stabilized space. This task is sometimes referred to as "deconvolution", and can be used, for example, to identify contributions from various tissues. Note: some groups have found that the mixing contributions may be more accurate if very lowly expressed genes across x and pure are first removed. We have not explored this fully. Note: if the pbapply package is installed a progress bar will be displayed while mixing components are fit.

Usage

```
unmix(x, pure, alpha, shift, power = 1, format = "matrix", quiet = FALSE)
```

Arguments

x	normalized counts or TPMs of the samples to be unmixed
pure	normalized counts or TPMs of the "pure" samples
alpha	for normalized counts, the dispersion of the data when a negative binomial model is fit. this can be found by examining the asymptotic value of dispersionFunction(dds), when using fitType="parametric" or the mean value when using fitType="mean".
shift	for TPMs, the shift which approximately stabilizes the variance of log shifted TPMs. Can be assessed with vsn::meanSdPlot.
power	either 1 (for L1) or 2 (for squared) loss function. Default is 1.
format	"matrix" or "list", default is "matrix". whether to output just the matrix of mixture components, or a list (see Value).
quiet	suppress progress bar. default is FALSE, show progress bar if pbapply is installed.

Value

a matrix, the mixture components for each sample in x (rows). The "pure" samples make up the columns, and so each row sums to 1. If colnames existed on the input matrices they will be propagated to the output matrix. If format="list", then a list, containing as elements: (1) the matrix of mixture components, (2) the correlations in the variance stabilized space of the fitted samples to the samples in x, and (3) the fitted samples as a matrix with the same dimension as x.

Examples

```
# some artificial data
cts <- matrix(c(80,50,1,100,
                1,1,60,100,
                0,50,60,100), ncol=4, byrow=TRUE)
# make a DESeqDataSet
dds <- DESeqDataSetFromMatrix(cts,
  data.frame(row.names=seq_len(ncol(cts))), ~1)
colnames(dds) <- paste0("sample",1:4)

# note! here you would instead use
# estimateSizeFactors() to do actual normalization
sizeFactors(dds) <- rep(1, ncol(dds))

norm.cts <- counts(dds, normalized=TRUE)

# 'pure' should also have normalized counts...
pure <- matrix(c(10,0,0,
                 0,0,10,
                 0,10,0), ncol=3, byrow=TRUE)
colnames(pure) <- letters[1:3]

# for real data, you need to find alpha after fitting estimateDispersions()
mix <- unmix(norm.cts, pure, alpha=0.01)
```

---

varianceStabilizingTransformation

Apply a variance stabilizing transformation (VST) to the count data

---

Description

This function calculates a variance stabilizing transformation (VST) from the fitted dispersion-mean relation(s) and then transforms the count data (normalized by division by the size factors or normalization factors), yielding a matrix of values which are now approximately homoskedastic (having constant variance along the range of mean values). The transformation also normalizes with respect to library size. The rlog is less sensitive to size factors, which can be an issue when size factors vary widely. These transformations are useful when checking for outliers or as input for machine learning techniques such as clustering or linear discriminant analysis.

Usage

```
varianceStabilizingTransformation(object, blind = TRUE, fitType = "parametric")

getVarianceStabilizedData(object)
```

Arguments

object	a DESeqDataSet or matrix of counts
blind	logical, whether to blind the transformation to the experimental design. blind=TRUE should be used for comparing samples in a manner unbiased by prior information on samples, for example to perform sample QA (quality assurance). blind=FALSE should be used for transforming data for downstream analysis, where the full use of the design information should be made. blind=FALSE will skip re-estimation of the dispersion trend, if this has already been calculated. If many of genes have large differences in counts due to the experimental design, it is important to set blind=FALSE for downstream analysis.
fitType	in case dispersions have not yet been estimated for object, this parameter is passed on to estimateDispersions (options described there).

Details

For each sample (i.e., column of counts(dds)), the full variance function is calculated from the raw variance (by scaling according to the size factor and adding the shot noise). We recommend a blind estimation of the variance function, i.e., one ignoring conditions. This is performed by default, and can be modified using the 'blind' argument.

Note that neither rlog transformation nor the VST are used by the differential expression estimation in DESeq, which always occurs on the raw count data, through generalized linear modeling which incorporates knowledge of the variance-mean dependence. The rlog transformation and VST are offered as separate functionality which can be used for visualization, clustering or other machine learning tasks. See the transformation section of the vignette for more details.

The transformation does not require that one has already estimated size factors and dispersions.

A typical workflow is shown in Section *Variance stabilizing transformation* in the package vignette.

If estimateDispersions was called with:

fitType="parametric", a closed-form expression for the variance stabilizing transformation is used on the normalized count data. The expression can be found in the file 'vst.pdf' which is distributed with the vignette.

fitType="local", the reciprocal of the square root of the variance of the normalized counts, as derived from the dispersion fit, is then numerically integrated, and the integral (approximated by a spline function) is evaluated for each count value in the column, yielding a transformed value.

fitType="mean", a VST is applied for Negative Binomial distributed counts, 'k', with a fixed dispersion, 'a': (2 asinh(sqrt(a k)) - log(a) - log(4))/log(2).

In all cases, the transformation is scaled such that for large counts, it becomes asymptotically (for large values) equal to the logarithm to base 2 of normalized counts.

The variance stabilizing transformation from a previous dataset can be "frozen" and reapplied to new samples. The frozen VST is accomplished by saving the dispersion function accessible with dispersionFunction, assigning this to the DESeqDataSet with the new samples, and running varianceStabilizingTransformation with 'blind' set to FALSE. Then the dispersion function from the previous dataset will be used to transform the new sample(s).

Limitations: In order to preserve normalization, the same transformation has to be used for all samples. This results in the variance stabilizition to be only approximate. The more the size factors differ, the more residual dependence of the variance on the mean will be found in the transformed data. rlog is a transformation which can perform better in these cases. As shown in the vignette, the function meanSdPlot from the package **vsn** can be used to see whether this is a problem.

Value

varianceStabilizingTransformation returns a DESeqTransform if a DESeqDataSet was provided, or returns a a matrix if a count matrix was provided. Note that for DESeqTransform output, the matrix of transformed values is stored in assay(vsd). getVarianceStabilizedData also returns a matrix.

Author(s)

Simon Anders

References

Reference for the variance stabilizing transformation for counts with a dispersion trend:

Simon Anders, Wolfgang Huber: Differential expression analysis for sequence count data. Genome Biology 2010, 11:106. http://dx.doi.org/10.1186/gb-2010-11-10-r106

See Also

plotPCA, rlog, normTransform

Examples

```
dds <- makeExampleDESeqDataSet(m=6)
vsd <- varianceStabilizingTransformation(dds)
dists <- dist(t(assay(vsd)))
# plot(hclust(dists))
```

---

vst *Quickly estimate dispersion trend and apply a variance stabilizing transformation*

Description

This is a wrapper for the varianceStabilizingTransformation (VST) that provides much faster estimation of the dispersion trend used to determine the formula for the VST. The speed-up is accomplished by subsetting to a smaller number of genes in order to estimate this dispersion trend. The subset of genes is chosen deterministically, to span the range of genes' mean normalized count.

Usage

```
vst(object, blind = TRUE, nsub = 1000, fitType = "parametric")
```

Arguments

object	a DESeqDataSet or a matrix of counts
blind	logical, whether to blind the transformation to the experimental design (see varianceStabilizingTransformation)
nsub	the number of genes to subset to (default 1000)
fitType	for estimation of dispersions: this parameter is passed on to estimateDispersions (options described there)

Value

a DESeqTranform object or a matrix of transformed, normalized counts

Examples

```
dds <- makeExampleDESeqDataSet(n=2000, m=20)
vsd <- vst(dds)
```

Index

∗ **package**
DESeq2-package, 3

bplapply, *8*, *47*

coef.DESeqDataSet, 4
collapseReplicates, 4
counts, *9*, *44*
counts,DESeqDataSet-method, 6
counts<-,DESeqDataSet,matrix-method *(*counts,DESeqDataSet-method*)*, 6

DESeq, *3, 4*, 7, *15, 16*, *22*, *28, 29*, *32, 33*, *35, 36*, *40*, *43–45*, *48*, *51*, *53*, *56*
DESeq2 *(*DESeq2-package*)*, 3
DESeq2-package, 3
DESeqDataSet, *3*, *7*, *9*, *13*, *21, 22*, *31*, *50*
DESeqDataSet *(*DESeqDataSet-class*)*, 10
DESeqDataSet-class, 10
DESeqDataSetFromHTSeqCount, *7*
DESeqDataSetFromHTSeqCount *(*DESeqDataSet-class*)*, 10
DESeqDataSetFromMatrix, *7*
DESeqDataSetFromMatrix *(*DESeqDataSet-class*)*, 10
DESeqDataSetFromTximport *(*DESeqDataSet-class*)*, 10
DESeqResults, *46*, *48*, *54*
DESeqResults *(*DESeqResults-class*)*, 12
DESeqResults-class, 12
DESeqTransform, *37*, *41*, *51*, *57*
DESeqTransform *(*DESeqTransform-class*)*, 12
DESeqTransform-class, 12
design,DESeqDataSet-method, 13
design<-,DESeqDataSet,formula-method *(*design,DESeqDataSet-method*)*, 13
design<-,DESeqDataSet,matrix-method *(*design,DESeqDataSet-method*)*, 13
dispersionFunction, 14, *17*, *56*
dispersionFunction,DESeqDataSet-method *(*dispersionFunction*)*, 14
dispersionFunction<- *(*dispersionFunction*)*, 14
dispersionFunction<-,DESeqDataSet,function-method *(*dispersionFunction*)*, 14
dispersions, 15, *18*
dispersions,DESeqDataSet-method *(*dispersions*)*, 15
dispersions<- *(*dispersions*)*, 15
dispersions<-,DESeqDataSet,numeric-method *(*dispersions*)*, 15

estimateBetaPriorVar, 15, *34*
estimateDispersions, *7*, *14, 15*, *19–21*, *51*, *56, 57*
estimateDispersions,DESeqDataSet-method, 16
estimateDispersionsFit, *18*
estimateDispersionsFit *(*estimateDispersionsGeneEst*)*, 19
estimateDispersionsGeneEst, *18*, 19
estimateDispersionsMAP, *18*
estimateDispersionsMAP *(*estimateDispersionsGeneEst*)*, 19
estimateDispersionsPriorVar *(*estimateDispersionsGeneEst*)*, 19
estimateMLEForBetaPriorVar *(*estimateBetaPriorVar*)*, 15
estimateSizeFactors, *7*, *23, 24*, *53*
estimateSizeFactors,DESeqDataSet-method, 21
estimateSizeFactorsForMatrix, *22, 23*, 23

fpkm, 24, *26*
fpm, *24, 25*, 25

getVarianceStabilizedData *(*varianceStabilizingTransformation*)*, 55

integrateWithSingleCell, 27

lfcShrink, *3*, *7–9*, 28, *40*, *47, 48*

makeExampleDESeqDataSet, 30

nbinomLRT, *4*, *7*, *9*, 31, *34, 35*, *40*, *44, 45*, *48*
nbinomWaldTest, *4*, *7–9*, *15*, *32*, 33, *40*, *44, 45*
normalizationFactors, *6*, *9*, *22*, *31*, *33*, 35, *53*
normalizationFactors,DESeqDataSet-method *(*normalizationFactors*)*, 35
normalizationFactors<- *(*normalizationFactors*)*, 35
normalizationFactors<-,DESeqDataSet,matrix-method *(*normalizationFactors*)*, 35
normalizeGeneLength, 36
normTransform, 37, *52*, *57*

plotCounts, *3*, 37
plotDispEsts, 38
plotDispEsts,DESeqDataSet-method *(*plotDispEsts*)*, 38
plotMA, *3*, 39
plotMA,DESeqDataSet-method *(*plotMA*)*, 39
plotMA,DESeqResults-method *(*plotMA*)*, 39
plotPCA, *3*, *12*, 41, *52*, *57*
plotPCA,DESeqTransform-method *(*plotPCA*)*, 41
plotSparsity, 42
priorInfo, 43
priorInfo,DESeqResults-method *(*priorInfo*)*, 43
priorInfo<- *(*priorInfo*)*, 43
priorInfo<-,DESeqResults,list-method *(*priorInfo*)*, 43

RangedSummarizedExperiment, *50*
register, *8*, *47*
removeResults *(*results*)*, 44
replaceOutliers, *8*, *9*, 43
replaceOutliersWithTrimmedMean *(*replaceOutliers*)*, 43
results, *3*, *4*, *7*, *9*, *11*, *12*, *27–29*, *32*, *34, 35*, *40*, *44*, 44, *54*
resultsNames *(*results*)*, 44
rlog, *12*, *37*, *41*, 50, *55–57*
rlogTransformation *(*rlog*)*, 50

shorth, *22*, *23*
show,DESeqResults-method, 52
sizeFactors, *6*, *21*, *22*, *31*, *33*, *35*, *36*
sizeFactors,DESeqDataSet-method, 53
sizeFactors<-,DESeqDataSet,numeric-method *(*sizeFactors,DESeqDataSet-method*)*, 53
summary,DESeqResults-method, 53

unmix, 54

varianceStabilizingTransformation, *12*, *14*, *18*, *37*, *41*, *50–52*, 55, *57*
vst, *3*, *51*, 57